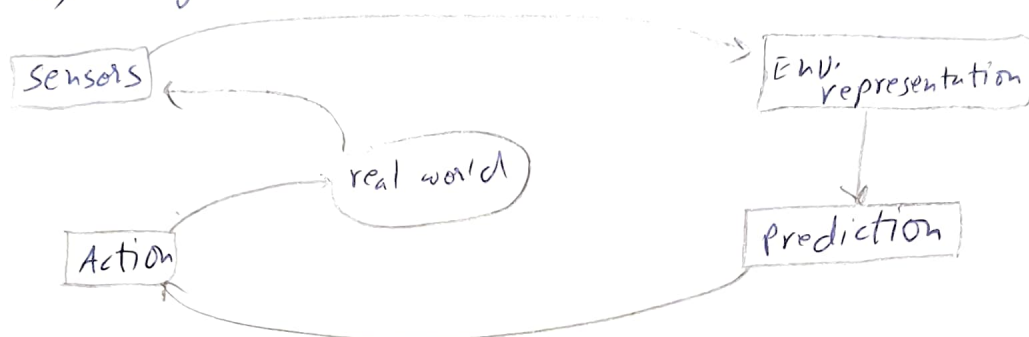


Chap1 :

Introduction

Perception in robotics refers to perceiving the surrounding environment and real world, to understand and take actions within it, using sensors and algorithms [H/W & S/W]



Examples of sensors: camera, lidar, infrared, ultrasonic

Acoustic sensors: vibration actuators & piezo-electric microphones

Internal state sensors: [pos, vel, acc], Inertial measurement units (IMUs), positioning [GNSS] Global Navigation satellite systems.

Environment Representation

- can be part of prediction & action algorithm

or

- be an individual module.

Based on the selected algorithm representation can be ~~part of~~ learned with prediction / action module.

Prediction Algorithm

- make predictions about the env.

- include

- Feature extraction
- object detection
- semantic segmentation
- Trajectory forecasting.

Action Algorithm

- act in response to sensed environment.

- include

- Localisation & Mapping
- planning & manipulation
- Trajectory planning
- Motion planning.

Human vision is the ability to interpret surrounding environment using light in the visible spectrum reflected by the objects in the environment.

Computer vision is the collection of algorithms and representations for automatic understanding from image, video, point clouds and other visual data.

Vision in robotics is embodied

CV [computer vision]

- static: does not move
- static input
- algo can run on a remote server.

Robotic Vision

- dynamic: can move & hence experience real world.
- dynamic & active i/p.
- algo. part of the world.

CV tasks

- Recognition: scenes, objects, humans & activities
- Detection & tracking: Flow estimation / Lane detection
- Segmentation: semantic segmentation, medical image segmentation
- Motion estimation & forecasting
- 3D reconstruction
- Depth estimation: Monocular Depth estimation
- Visual search

* Facial keypoint detection [Face detection]

* gesture recognition

* Emotion recognition

* Text spotting

Why is CV necessary?

- Automate processes/reduce human effort
- Perception of autonomous agents
Ex: Automated driving
- support visually impaired
- Human computer interaction
- MetaVerse/Augmented Reality

Applications

- Face recognition from image & video
- Automated driving:
 - * lane detection
 - * traffic light detection
 - * optical flow estimation
 - * depth estimation
 - * object detection.
- Trajectory forecasting & planning.

Difficulties/challenges:

- Real world more than captured images.
Images are only an abstraction of the world
- Inverting image formation process is not straightforward & impossible without prior ~~knowledge~~ information. [Ill posed problem]
 - * clutter
 - * occlusions
 - * intraclass variation
- Variation in
 - * illumination
 - * obj pose
 - * Image viewpoint
- context is important: context is important for interpretation
- highly complex tasks:
 - * Thousands of object categories
 - * Millions of different camera models
 - * Thousands pixels per image.

Human vision is complex because

- humans learn to understand visual input over a long time.
- vision is not the only sense.

Build Computer vision algorithms that mimic humans. ~~behavior~~

- understand the world & model it
- not possible to model all kinds of objects & scenes

CV v/s Human vision

- Advantages of CV algorithms

- * Repentability: results on same ip stay same if computed several times.
- * Tractability: algorithmic design includes knowledge about criteria that led to an interpretation
- * No fatigue: performance does not degrade with time.
- * No shift of viewpoint: unless being part of the algorithm. Decisions not influenced by results from previous decisions or external influences.
- Simulate processes of early vision (retinal/primary visual cortex) for data processing.
Results in: image enhancement / low-level image features.
- Develop approaches from cognitive psychology [segmentation: decomposition into components]
 - * computation of low-level image features
 - * grouping for segmentation
 - * matching with prototypes
 - * recognition by components.
- CV most successful for early vision tasks
 - * noise reduction / edge detection, corners, lines & end points detection.
 - * stereo vision
 - * motion detection

Challenges in mimicing human vision

- early-vision steps may not be optimal as subsequent analysis in visual perception is unknown.
- Knowledge in cognitive psychology about visual perception
 - * is qualitative than quantitative, hence difficult to map using an algorithm.

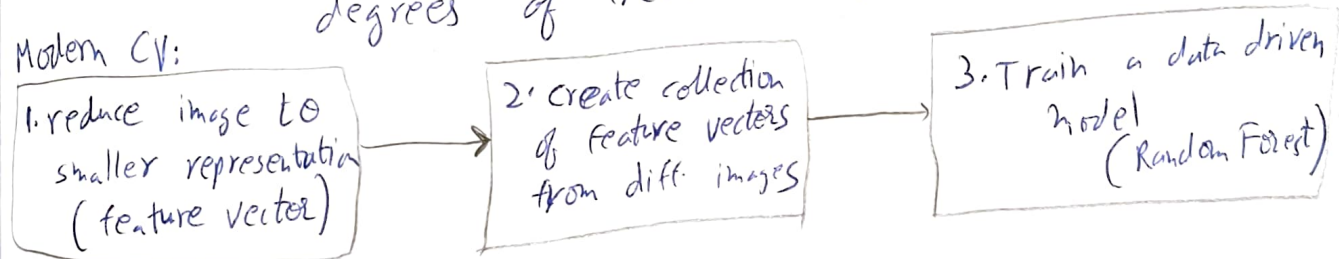
* relates to specific subjects that are part of a solution but not the solution itself.

- human vision may provide **useful strategies** but cannot be a **complete model** for an **algorithmic solution**.
- classic CV has been based on idea of mimicking human vision.
- works well for ex: image de-noising.
- cannot generalise through to any kind of problem.

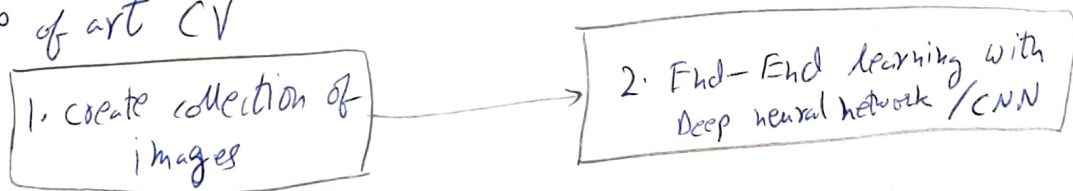
I **Data driven algorithms**

- Algorithms / models with learnable parameters.
- large training data-sets employed for learning.
- separate tasks into smaller subtasks
 - * single decision under constrained conditions rather than a general vision system.
 - * assume some kind of hierarchy in visual object recognition, with elements of hierarchy having fewer degrees of freedom.
↳ subtasks

Modern CV:



State of art CV



II Task decomposition:

Robotic Perception:

- object detection
- semantic segmentation
- human pose estimation

Human Pose estimation

- within-task decomposition i.e. body parts.

— X —

Image is represented using matrices with multiple channels for color images and with single channel for grayscale images.

- Uses discrete image width & height.
- " " channel values. ex: 0 to 255
- Color spaces: RGB, CMYK

Image digitization converts information into digital format/computer-readable format.

Images are captured by sensors and represented the world using pixels.

Image Noise:

- capturing same image not possible even with static sensor
- no identical images due to noise

- Image denoising can be done using linear or non-linear operators.

noise types

- Gaussian Noise: intensity variation based on PDF equal to normal distribution
- Impulse Noise: random occurrence of white pixels
- Salt & Pepper Noise: random occurrence of Black & white pixels.
- Quantization Noise
- Periodic noise from electrical or electromechanical interference during image capturing.

Sources of Noise

1. Sensor noise
 - i) illumination
 - ii) Temperature
 - iii) Electric circuit noise
2. Sensor size: larger the size, more the noise captured.

Filtering is the estimation of expected value of each pixel.

- Estimation can be done by:
- averaging pixel values in neighbourhood
 - pixel values in neighbourhood are assumed to be constant
 - **Neighbourhood size**: is a compromise b/w a good estimate (large neighbourhood) and validity of assumption of constancy (small neighbourhood)
 - overall filtering realized by convolution.

Filter = mask = kernel

↳ Filtering modifies image pixels based on an operator.
 output of filtering = smoothed image [smoothing]

Assumptions when filtering

- noise process is independent b/w pixels
- pixels within neighbourhood expected to be similar.

Example of **1D filtering**: Weighted moving average
 either i) equally weighted
 or
 ii) non-equally weighted.

$$V_3 = \frac{1 \cdot V_1 + 1 \cdot V_2 + 1 \cdot V_3 + 1 \cdot V_4 + 1 \cdot V_5}{1+1+1+1+1} ; V_3 = \frac{1 \cdot V_1 + 2 \cdot V_2 + 4 \cdot V_3 + 2 \cdot V_4 + 1 \cdot V_5}{1+2+4+2+1}$$

In **2D** use a sliding window.

Ex of 2D kernel equally weighted

- **hyperparameters**: kernel size & stride

1	1	1
1	1	1
1	1	1

→ kernel of size 3x3
 Non-uniform kernels are also possible.

slide window → element wise multiplication of filter with image patch

Mean filtering → used for noise reduction.

Linear filter vs Non-linear filter?

given kernel size & image size calc. resultant image size with padding / without padding?

Filtering useful for:

- 1) Image enhancement: denoising, resize
- 2) Feature extraction: edges
- 3) Detect patterns; ex: template matching.

Correlation Filtering

Nonweighted averaging window of size $2k+1 \times 2k+1$

$$G[i, j] = \frac{1}{(2k+1)^2} \sum_{u=-k}^k \sum_{v=-k}^k F[i+u, j+v]$$

Uniform weight of 1 to each pixel

general eqn.

$$G[i, j] = \sum_{u=-k}^k \sum_{v=-k}^k \underbrace{H[u, v]}_{\text{kernel weights}} F[i+u, j+v]$$

What is similar operation to cross correlation?

$$F[i, j] \rightarrow [H[u, v]] \rightarrow G[i, j]$$

$$G = H \otimes F \rightarrow \text{cross correlation}$$

Replaces each pixel with linear combination of neighbouring pixels.

Gaussian Filter

$$h(u, v) = \frac{1}{2\pi\sigma^2} e^{-\frac{u^2+v^2}{2\sigma^2}}$$

u & v are coordinates of kernel
 $h(u, v) \rightarrow$ value of kernel at those coordinates.

- it is a linear filter
- removes high-Freq. components (low-pass filter)
- influence of close pixels more than pixels far away.
- Results in a blurred image.

how to choose width & height of the kernel?

Convolution

- same as correlation but kernel is flipped in both directions.

$$G[i, j] = \sum_{u=-k}^k \sum_{v=-k}^k H[u, v] F[i-u, j-v]$$

$$G = H * F \rightarrow \text{convolution.}$$

- 1) Why is kernel flipped for convolution?
- 2) Diff. b/w conv. & correlation?

$u \rightarrow \text{row}$
 $v \rightarrow \text{col}$

kernel (u,v)

-1, -1	-1, 0	-1, 1
0, -1	0, 0	0, 1
1, -1	1, 0	1, 1

Correlation
 $F(u,v)$

-1, -1	-1, 0	-1, 1
0, -1	0, 0	0, 1
1, -1	1, 0	1, 1

Convolution
 $F(u,v)$

1, 1	1, 0	1, -1
0, 1	0, 0	0, -1
-1, 1	-1, 0	-1, -1

indicates which pos. to multiply
for correlation. A convolution
when filter is not flipped for
convolution.
If flip of kernel/F is done then
usual multiplication is enough.

Median Filtering

- Non-linear image filtering.
- no kernels but sliding window/neighbourhood operation
- Median is extracted from range of values within sliding windows.
- removes outliers / spikes / pixels with distant values.
- replaced by pixel values that are like neighbourhood pixels.
- Preserves edges & removes noise.
- Works best for salt & pepper noise.

Is it an
expensive filter
operator?

Boundary Treatment

- op image has lesser dimensions.
- use padding to preserve image dimensions.
 - * can pad with zeros or other constants
 - * can mirror boundary values for padding.

Separable filters

- Decompose filters into 2 filters. Ex: 3×3 filter into 1×3 followed by 3×1

- reduces computational cost

image $\rightarrow N \times M$

$O(N \times M \times m \times n) \rightarrow$

filter $\rightarrow m \times n$

$O(M * N * (m+n))$

$$\frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix} = \frac{1}{3} \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} \cdot \frac{1}{3} \begin{bmatrix} 1 & 1 & 1 \end{bmatrix}$$

Bilateral Filter

- Non-linear & edge preserving filter
- spatial domain: takes into account spatial dist. b/w centred pixel & neighbouring pixels.
- Image domain: takes into account intensity diff. b/w centred pixel & neighbouring pixel.
- weighing scheme for influence of spatial dist & intensity diff.

parametrised by

- $f_{sd} \rightarrow$ spatial domain weighing [usually 2D gaussian kernel]
- $g_{sr} \rightarrow$ intensity domain weighing [usually 1D gaussian]
- $\sigma_{sd} \rightarrow$ defines size of gaussian kernel
- $\sigma_r \rightarrow$ minimum amplitude of edge

parametrised by

Bilateral vs gaussian filter?

$$I_p = \frac{1}{w_p} \sum_{q \in S} I_q f_{sd}(\|q-p\|) g_{sr}(|I_q - I_p|)$$

$I_p \rightarrow$ image pixel ~~at~~ at p

$$w_p = \sum_{q \in S} f_{sd}(\|q-p\|) g_{sr}(|I_q - I_p|)$$

neighbourhood pixels indexed by q ; $S \rightarrow$ neighbourhood pixels.

$\|-\| \rightarrow$ euclidian dist
 $|-| \rightarrow$ absolute diff.

$P \rightarrow$ padding

$H_{in} \times W_{in}$: input matrix

$H_f \times W_f$: filter

$H_{out} \times W_{out}$

$$H_{out} = (H_{in} - H_f + 2P) + 1$$

$$W_{out} = (W_{in} - W_f + 2P) + 1$$

Tactile texture: refers to actual feel of surface or physical surface quality of an object.
can be felt through touch.
Ex: roughness of sandpaper / smoothness of glass.

Visual texture: refers to illusion or appearance of texture in an image.
perceived through sight & not touch.
This texture can be created by manipulating colors, light & shadows.
Ex: picture of tree bark.

Texture can be **natural & artificial**. Ex: calibration patterns. → used to fine-tune devices like cameras & displays / printers.

Texture **can be extracted** from images using set of operations / functions applied to the image.

Texture **provides** information of spatial arrangement of color/intensities in an image.

Texture identification
pixels → segments

Transforms image from
pixels to segments.

What is **image texture**?

structural approach

- set of **texels** in a structured & repetitive way.

statistical approach

- quantitative measure of intensity arrangement.

Types of textures

- Natural/artificial
- regular patterns
- random patterns

Contribution of texture

- for object & scene description
- can be used for object detection, tracking and recognition
- similar shaped objects can be distinguished based on their texture.

Texel (Texture Element) is smallest unit of texture map. Each texel has color, position & size.

↳ determined by texture resolution

How to detect an object from texture?

Texture in an image is some repeated local patterns.

Procedure

1. create filters to describe objects
2. apply these filters across image & consider response magnitude for finding matching filter
3. collect statistics after application.

Example

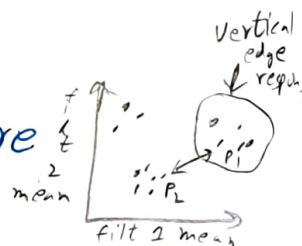
1. **Gabor filters**: human designed filters for extracting texture.
 - Detect frequency & orientation like Human Visual System.

- i) apply filter across image
- ii) slide window over filter response & collect statistics
Ex: mean ~~of~~ value per patch.
- iii) Summarise statistics for all windows.

Each gabor filter corresponds to an image feature

- cluster the feature responses

- each cluster has semantic meaning Ex: edges vertical/horizontal, corners, lines

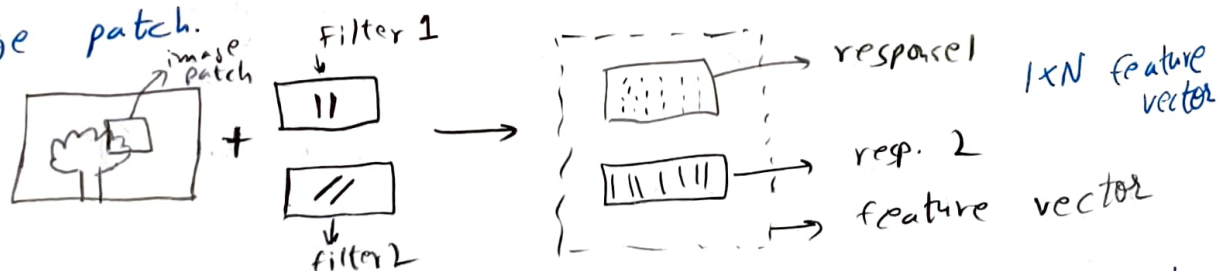


When we have low compute power we use such filters: Ex - rovers

- distance between clusters indicate dissimilarity. Euclidean distance is a good reference.

Filter Banks: is collection of different filters, designed to detect textures.

- 2 filters result in a feature vector with 2 elements for each image patch.



N-filters result in N-dimensional feature vector for each image patch.

How many filters are sufficient?

- Design of filter banks with different orientation & scale.
- Design filters ~~bank~~ with different orientation & scale.
- Use convolution operation to efficiently extract filter responses.

- Goals
 - detect all possible patterns that describe objects
 - distinguish objects based on filters.

What are modern filters in machine learning? → CNN learn the filter banks.
 Which filter is right one to keep? → Neural Networks
 keep everything as we do not know which one to keep/throw away.

CNN learn the filter banks:

Ex: Layer 1 → input layer
 " 2 → edges
 " 3 → shapes
 " 5 → faces/noses

2. Leung-Malik (LM) filter bank

48 filters — Multiscale & Multi-orientation

4 gaussian + 8 LOG + 1st & 2nd derivatives of gaussians (36 filters at 6 orientations & 3 scales)

Detects — edge features
 — bar features
 — spot features

3. Schmid filter bank: 13 rotationally invariant filters

gives a good response irrespective of orientation of object.

4. Maximum Response (MR) filter bank

Pixels are represented by the feature vectors.

sliding window approach

1. Extract M image patches from sliding window
2. Per image patch
 - compute N feature vectors by convolution with N -filters.
 - compute mean of response map
 - Replace pixel value with the N computed values
3. Image is replaced with set of response maps.

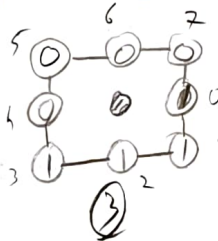
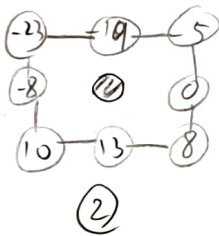
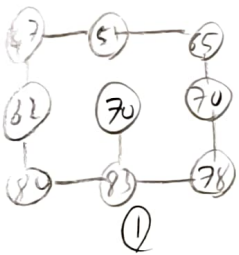
Local Binary Pattern

- used for facial recognition
- Uses/detects local spatial patterns & grayscale contrast.
- sliding window & thresholding WRT center pixel value of window
 $\hat{Z}(\text{histogram})$
- robust to monotonic grayscale changes, ex: illumination variation.
- gives same response despite illumination.

$$LBP_{P,R} = \sum_{p=0}^{P-1} s(g_p - g_c) 2^p \quad ; \quad s(x) = \begin{cases} 1 & \text{if } x \geq 0 \\ 0 & \text{o.w} \end{cases}$$

1. sample $\rightarrow$ 2. difference $\rightarrow$ 3. threshold

4. multiply by powers of 2 & sum



$$\begin{aligned} & 1 \times 2^0 + 1 \times 2^1 + 1 \times 2^2 + 1 \times 2^3 \\ & + 0 \times 2^4 + 0 \times 2^5 + 0 \times 2^6 + 0 \times 2^7 \\ & = 15 \end{aligned}$$

④

- applications: texture classification
- Face & expression detection [local + global descriptors]
- Extension: Median Robust Extended LBP: compare regional image medians instead of raw pixels.

Image sampling

Subsampling is process of reducing the resolution of an image by selecting fewer pixels. It decreases size of the image.

Upsampling is reverse of sub-sampling - it increases resolution of an image by adding more pixels.

subsampling causes loss of detail & may result in aliasing.

Upsampling can result in blurred or pixelated image.

Distortion artifacts can be seen when subsampling an image. [moiré patterns]

spatial anti-aliasing: technique of minimizing distortion artifacts when sub-sampling an image.

- remove higher frequency signal components & then resample.

continuous time signals [analog sig.] → how world functions [sound, light]

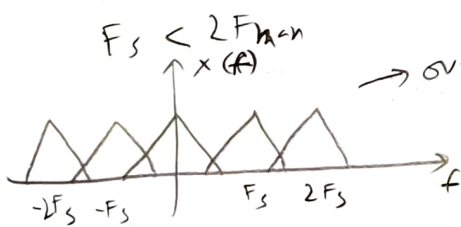
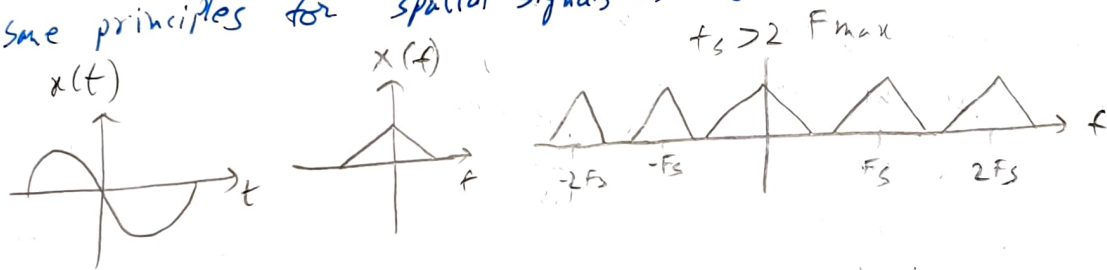
Discrete time signals [digital sig.] → how systems function. [camera/hard drive]

Nyquist theorem: A bandlimited continuous-time signal can be sampled with frequency F_s & can perfectly be reconstructed if number of samples per second is atleast twice highest freq. F_{max} present in waveform.

$$F_s \geq 2 F_{max}$$

shannon: To reconstruct all frequencies in a CT signal, it has to be sampled twice the highest freq. (F_{max}) present in sig.

Same principles for spatial signals as well.



→ overlapped regions cause aliasing
make signals not indistinguishable.
cannot be perfectly reconstructed

Anti-Aliasing techniques

- $F_s > 2 F_{max}$: sample with such freq.
- apply LPF to limit F_{max}

Moiré patterns are interference patterns that occur due to ~~aliasing~~ ^{aliasing}.

Image Representation

Pixel based representation / raw pixel representation is not always advantageous.

Not a good representation due to the challenges

- Lack of invariance in illumination changes, translation, rotation or scale changes.
- Image context not equally important

Pixel-based image modelling

Detection: similar objects can still look different based on metric norm for comparison.

Description: different description of same object based on view/illumination.

Matching: matching same object based on raw pixel comparisons impossible.

Local features cover a small image region and have/represent particular image structures like edge/corner/blob/point, which are mostly detected based on image gradients instead of directly using image intensities.

Several local features may exist per picture.

- image patch of local feature is distinctive from its neighbourhood
- local feature is described by a feature vector. ex: gradients for given patch (patch location + size)
- Each local feature contributes to describe a complete image.

Desired Properties of local features

1. Locality: feature occupies small region
 - robust to occlusion and clutter
2. Repeatability: same feature of an object is detected in different images regardless of geometric & photometric transformations
 - skewed/rotation
 - illumination/contrast
3. Distinctiveness: distinct description for each vector. [description $\rightarrow$ gradients/histogram]
4. Compactness and computationally efficiency
 - fewer features than image pixels
 - light-weight computation of feature & its description.

To find local features irrespective of scale of the object 2 methods could be used

1. Fix filter size & rescale the image or
2. Modify size of filter to get the local features across images.

How does scale of object ^{in the image} impact finding of local features?

Keypoints / interest points and features : are good image regions to detect and track.

These keypoints can be used for CV tasks for correspondence finding between 2 images

- image search
- 3D reconstruction
- object tracking, detection & recognition.

← How to use interest points for CV tasks?

IMAGE stitching / Panorama

1. Extract features/keypoints b/w 2 images
2. Feature matching
3. Image alignment

Method of keypoint matching:

1. Find distinctive point for database & store them
 2. " " points " query image
 3. Define region of interest for keypoint & extract descriptor
 4. Compare query descriptor with database descriptors [matching]
the feature vectors
- working with quantised values can make processing/matching faster.
operator for matching quantised values also closer to mc/assembly language.
- Detect repeatable & distinctive keypoints for keypoint matching.
↳ represent particular & relevant content

Corner Features

Corner is a point where 2 edges meet. It is a stable feature that can be used to describe an image.

- can observe large variations in intensity of the neighbourhood of the corner point in all directions. [look at gradient change, ~~edge~~ sliding window in x & y direction to look at pixel values]

Idea: consider a small window around a pixel & look at intensity values. For a corner feature shifting window in any direction results in a large intensity change. In practice select neighbouring windows in a sliding window fashion.

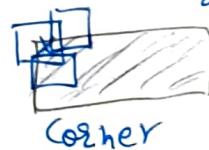
no change in any dir →



flat region



Edge



Corner

Harris Corner detector

$$E(u, v) = \sum_{x, y} w(x, y) [I(x+u, y+v) - I(x, y)]^2 \rightarrow \text{Harris response function}$$

$\uparrow$ window func. $\uparrow$ shifted I $\uparrow$ reference I

- Uses window average squared change of intensity.
 - given a reference x, y shift the window around it and compute intensity differences.
 - window function weighs the difference contribution. Ex: Gaussian window.
1. const. patches: I difference will be around 0
 2. distinctive patches: difference is large
 3. corner features: large differences are expected.

Derivation

2nd order approx

$$f(x+u, y+v) = f(x, y) + u f_x(x, y) + v f_y(x, y) + \frac{1}{2!} [u^2 f_{xx}(x, y) + 2uv f_{xy}(x, y) + v^2 f_{yy}(x, y)] + \dots$$

1st order approx

- 1st order approximation is enough
- What is limitation of using higher order approximations? $\rightarrow$ computations increase & require more compute power

$$E(u, v) = \sum_{x, y} w(x, y) [I(x+u, y+v) - I(x, y)]^2$$

$$\sum_{x, y} [I(x+u, y+v) - I(x, y)]^2 \approx \sum_{x, y} [I(x, y) + u I_x + v I_y + I(x, y)]^2$$

$$= \sum_{x, y} [u^2 I_x^2 + v^2 I_y^2 + 2uv I_x I_y]$$

$$= [u \ v] \left(\sum_{x, y} w(x, y) \begin{bmatrix} I_x^2 & I_x I_y \\ I_x I_y & I_y^2 \end{bmatrix} \right) \begin{bmatrix} u \\ v \end{bmatrix}$$

assuming small pixel shifts (u, v) & bilinear approximation

$$E(u, v) \approx [u \ v] M \begin{bmatrix} u \\ v \end{bmatrix}$$

$$M = \left(\sum_{x, y} w(x, y) \begin{bmatrix} I_x^2 & I_x I_y \\ I_x I_y & I_y^2 \end{bmatrix} \right)$$

$w(x, y) \rightarrow$ computes a weighted sum
 $M \rightarrow$ products of gradients
 2×2 M matrix computed using image derivatives.

Example of image derivatives: Sobel operator

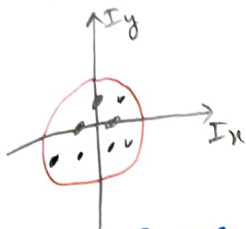
Sobel operator approximates the gradients in x & y direction.

Gradient $x: I_x$
 " $y: I_y$

plot of the derivatives



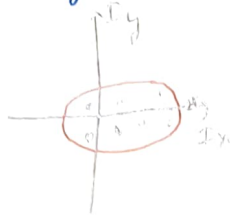
flat region



both I_x, I_y are close to zero

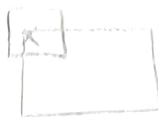


edge

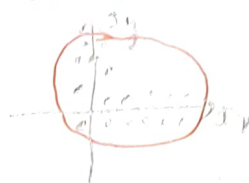


- only I_x or I_y values larger than zero depending on direction of edge.

- No corner info



corner



- I_x & I_y values larger than zero.
- corner info present.

→ fitting ellipses *

Fitting an ellipse *

$$M = X \begin{bmatrix} \lambda_1 & 0 \\ 0 & \lambda_2 \end{bmatrix} X^T$$

- λ_1 & λ_2 are small and similar

- λ_1, λ_2 fit an ellipse around our data

- λ_1 is large and λ_2 is small (and the opposite)

- λ_1, λ_2 are both large.

for Harris corner detection

- region classification based on eigen values

- amount of intensity change in principle orthogonal gradient directions

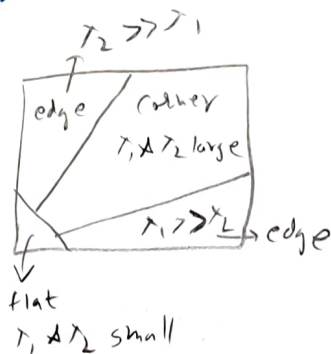
- Measure of corner response:

$$* R = \det(M) - k \text{trace}(M)^2$$

$$* \det(M) = \lambda_1 \lambda_2$$

$$* \text{trace}(M) = \lambda_1 + \lambda_2$$

$$* k \rightarrow \text{empirical const (0.9 - 0.95)}$$



step 1: seek points with $R > \text{threshold}$

step 2: Apply Non-maximum suppression to keep 1 maximum per neighbourhood

Invariant function: does not change its value regardless of the past arguments.

covariant function: changes its values according to the sampled parameters/arguments of the transformation.

Give examples of Invariant & Covariant transformations?

↓
affine intensity shift $I(x,y) \rightarrow I(x,y) + b$

Partially invariant $\rightarrow$ affine intensity scaling $I(x,y) \rightarrow aI(x,y)$

affine transform $\rightarrow$
is a linear mapping that preserves points, straight lines and planes.

Derivatives are invariant to intensity shifts.

* **Translation** in image when using Harris corner detector

- corner location is covariant WRT translation

- ~~corner~~ response is invariant

- image derivatives are invariant.

Invariance & covariance
WRT rotation and shift
✗

edge detector needs to be more invariant/covariant?

* **rotation**

- ~~corner~~ location is covariant

- corner response is invariant

- response map ~~changes~~ as we rotate i.e. ellipse rotates.
as image rotates but the shape (i.e. eigenvalues) remains same.

* **Scaling**

- corner location is not covariant WRT scale.

- shift window size is affected from the scale.

- corners can be classified as edges at different scales.

How to detect corners at different scales?

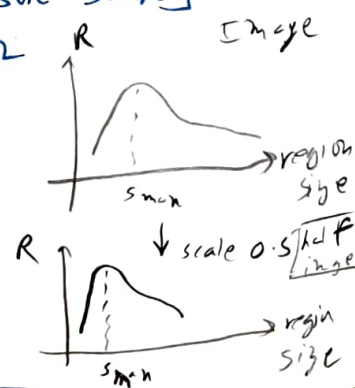
- Find scale that maximises λ_1 and λ_2 [characteristic scale]

- corner response measure $R = \det(H) - k \cdot \text{trace}(H)^2$

- collect responses at different image resolutions

- good scale function should have 1 sharp peak

- corner density is influenced by scale of image.



1st order image derivatives (chap-5)

sharp changes in image intensities indicate min/max of the 1st order derivative. Ex: used for extracting edges & corners [Ex: Harris corner detector]

2nd order image derivative

We look at the zero crossings of the 2nd derivative.

At the zero crossing of the 2nd derivative we find min/max of 1st derivative. [i.e. check peak of 1st derivative at a crossing of 2nd derivative]

To obtain 2nd order derivatives use kernels like

$$\begin{bmatrix} 1 & -2 & 1 \end{bmatrix}$$

$$\begin{bmatrix} 1 \\ -2 \\ 1 \end{bmatrix}$$

→ central differences approximations

Laplacian of Gaussian

It is a 2D isotropic measure of 2nd order image derivative.

- applies on the same way throughout the image
- measurement is independent of direction of observation.

$$L(x,y) = I_{xx} + I_{yy} = \left(\begin{bmatrix} 1 & -2 & 1 \end{bmatrix} + \begin{bmatrix} 1 \\ -2 \\ 1 \end{bmatrix} \right) * I = \begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix} * I \rightarrow \text{Laplacian filter.}$$

Differences/advantages of ~~both~~ 2nd order derivatives

$$\Rightarrow \nabla^2 I(x,y)$$

- Laplacian has more details
- Sobel skips some details
- Laplacian is more computationally expensive.

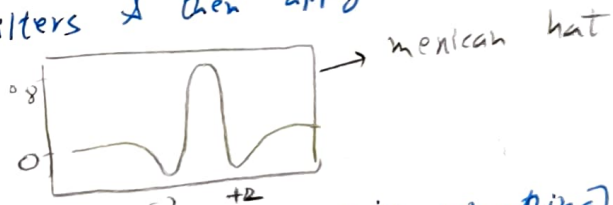
The 2nd derivative approximation is sensitive to noise.

To reduce noise use Gaussian smoothing.

1. Apply Gaussian smoothing
2. Apply Laplacian filter

} both filters can be applied using convolution
More efficient way is to convolve both the filters & then apply 1 filter at the end.

$$G(x,y) = -\frac{1}{\pi \sigma^4} \left[1 - \frac{x^2 + y^2}{2\sigma^2} \right] e^{-\frac{x^2 + y^2}{2\sigma^2}}$$



→ influences quality of detections [σ is scale of Gaussian smoothing]

- smoothens the image more heavily. suppresses small scale noise but also loss of fine edges.
- Shallow edges & fine textures are smoothed out & may not be detected
- edges are less precisely localized.
- increasing σ reduces detection of ~~noise~~ edges caused by noise.

Limitation of LoG: Detection of fake edges
[Canny edge detector more robust]

Difference of Gaussian (DoG)

- Approximate LoG with difference of 2 Gaussians.

$$\nabla^2 G_\sigma = G_{\sigma_1} - G_{\sigma_2} \quad ; \quad \sigma_1 = \frac{\sigma}{\sqrt{2}} ; \sigma_2 = \sqrt{2}\sigma$$

- More efficient to implement

- O/p of DoG $\approx$ o/p of LoG : o/p is very similar.

Method [Marr-Hildreth Algorithm $\rightarrow$ used for blob detection]

1. convolve image with 2 Gaussian kernels with σ_1 & σ_2
2. subtract both the Gaussian filtered images.

$\sigma \rightarrow$ scale

σ_1 & σ_2 affect the DoG quality.

$$\sigma_1 = \frac{\sigma}{\sqrt{2}} \quad \sigma_2 = \sqrt{2}\sigma$$

- small σ gives better edges but gives edges even for noise.

Response of LoG/DoG

1. Zero at a long dist from edge
2. +ve close to one side of edge
3. -ve " " other " " "
4. zero on the edge.

zero crossings of LoG/DoG form contours.
 σ affects contour detection.

Blob detection

Blob is a region in an image which is either darker/brighter than their surroundings

LoG/DoG can be used for blob detection.

Multiscale blob detection

$L(x, y, t)$ achieves maximum at centre of the blob given that the provided scale is correct.
 $\rightarrow$ scale [zoom on the image]

Automatic scale selection for LoG/DoG can be used to find max of $L(x, y, z)$ or find blobs in the right scale.

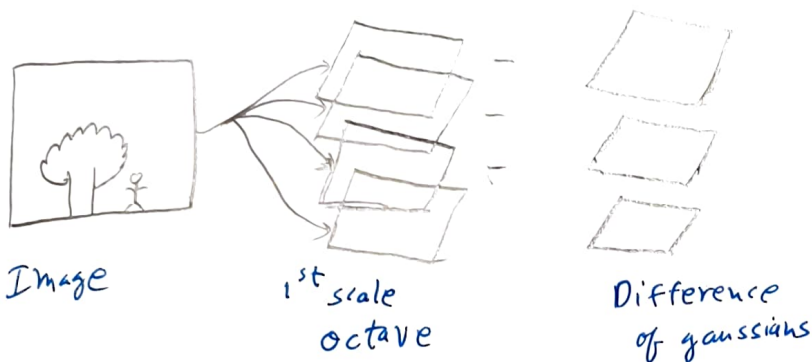
Method: Look for scale space maxima/minima.

The point x, y, z is a blob if its value is larger than its neighbours.

Bright blob: value larger than neighbours

Dark blob: value smaller " "

Illustration.



Use visual examples & discuss how to detect at different scales?

10 images

→ scale space
= $10 \times L \times W \times h$

→ width & height of original image is kept.

check local neighbourhood for maxima & minima values.

→ check not only within scale but across scales for extremum values.

It is equivalent to searching in a volume and not a 2D space.

→ Response will have extremum value at the right scale.



→ Threshold the blobs to filter out the bad blobs.

Hessian Detector

Instead of depending on 2nd order moments M & response R ,

we compute the Hessian matrix & determinant.

$$H(I) = \begin{bmatrix} I_{xx} & I_{xy} \\ I_{xy} & I_{yy} \end{bmatrix} \Rightarrow \det(H(I)) = I_{xx} I_{yy} - I_{xy}^2$$

↳ 2nd order partial derivatives of image intensities.

- Not scale invariant

- Scale space search required to find max. response

- How to apply to different scales? → use scale space representation

1. gaussian smoothing at different scales

2. Calc. Hessian at each scale

3. compute determinant of Hessian

4. Non-max. suppression in both spatial & scale dimensions.
5. combine results ^{features} across scales.

compute determinant of Hessian $[H_{\theta}]$ at different scales.
blobs are detected at right scales.

$$\det(HL) = t^2 (L_{xx}L_{yy} - L_{xy}^2)$$

$$L(x, y, t) = G(x, y, t) I(x, y)$$

L scaled image at scale t & location x, y .

$$G(x, y, t) = \frac{1}{2\pi t} e^{-\frac{x^2+y^2}{2t}}$$

$$x^*, y^*, t^* = \underset{x, y, t}{\operatorname{argmax}} (\det(HL)(x, y, t)).$$

hint ^{for} Analysing pictures with blobs

1. noisy
2. missing some info/blobs
3. False +ve blobs [shows blobs even when they are absent]
4. Accurate but misses few blobs
5. Detection of more n.o. of blobs
6. Blobs properly fitted.

Photometric problems: more illumination reduces contrast/gradient.
hence transforms might not give same response.

Sol: 1) standardization / normalisation

2) change color space: CMYK / where illumination is on other channel.

Geometric transformations

- translation
- rotation
- scale

photometric transformations
- illumination changes

→ both geometric & photometric transformations occur together most of the time.

→ matching features b/w 2 images useful for

- panorama
- tracking & 3D reconstruction
- linear transformations & solve sys. of linear eqns. for applications like speed/distance.

→ motivation: same feature should be detected regardless of the transformation.

SIFT [Scale Invariant Feature Transform] Chap-6

Algorithm includes feature detection → description → matching.

1. keypoint detection: scale & orientation invariant.
2. keypoint description: histogram of local image gradients & orientations.
3. keypoint matching: Efficient distance measurement w/ key points.
simplest operator AND can be fast for matching.

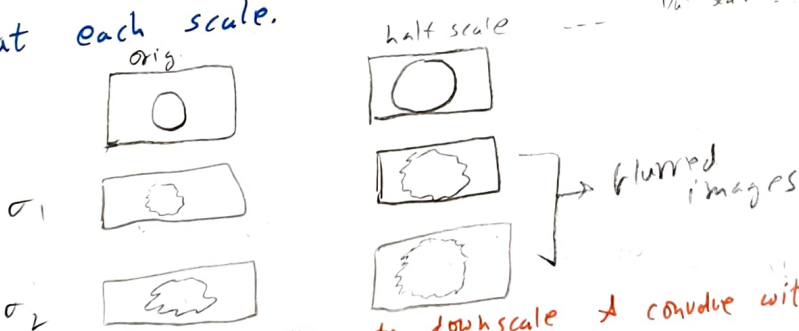
steps

- 1) scale space formation
- 2) keypoint detection
- 3) " orientation estimation
- 4) " descriptor extraction.

I Scale space formation

scale space is a collection of images formed from 1 image by rescaling and gaussian filtering.

How: Downscale image N times and apply gaussian filters with 2σ 's at each scale.



Use 4 octaves, i.e. downscale image 4 times.
How many times to downscale & convolve with gaussian filter?

Use 5 different filters/sigma for gaussian filtering.

II Keypoint Detection

- 1) Use DoG to extract image features.
Use DoG within each octave by subtracting the gaussian flurred images.
5 flurred images $\Rightarrow$ 4 difference images [dog]
- 2) Look for important keypoints by finding local maxima/minima [initial outlier rejection]. Compare using local window, in spatial and scale space domain. Pick up points with highest/lowest DoG response.

3) Key point localization

Key points now after local max/minima are scale-invariant but not necessarily robust to noise.

— initial outlier rejection: remove local contrast points.

Localize Key points / Location of keypoints found by.

1. 2nd order Taylor series approximation of DoG scale space.
2. Take derivative & solve for extrema of this function.
1st calculate derivative of DoG $\left[\frac{\partial \text{DoG}}{\partial x} \quad \frac{\partial \text{DoG}}{\partial y} \quad \frac{\partial \text{DoG}}{\partial z} \right]$
Then " second derivative $\begin{bmatrix} \text{DoG}_{xx} & \text{DoG}_{xy} & \text{DoG}_{xz} \\ \text{DoG}_{yx} & \text{DoG}_{yy} & \text{DoG}_{yz} \\ \text{DoG}_{zx} & \text{DoG}_{zy} & \text{DoG}_{zz} \end{bmatrix}$
→ 2nd derivative is more robust to noise

finally calc. $\hat{x} \rightarrow$ is local extrema point

3) Go back to DoG & calc. $\text{DoG}(\hat{x})$

4) If value < 0.03 discard the point. Keep the remaining points

III Keypoint Orientation Estimation [To make keypoints rotation invariant]

Assign orientation to each of the remaining keypoints.

Now we rely on image gradients & Not DoG.

1. calc. magnitude $m(x,y)$
2. calc. orientation $\theta(x,y)$
3. Based on $m(x,y)$ and $\theta(x,y)$ create direction weighted histogram.

We use image gradients as the points are now reliable, we can calc. actual gradients at each point instead of using approximations.

creation of weighted direction histogram:

1. create bins for angles ex: 10 degrees bins [0-10, 10-20 ...]

2. Assign $\theta(x,y)$ to a bin
3. Weigh them based on magnitude

given a partial histogram fill 2 / 3 bins.

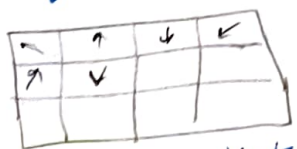


image gradients



Angle weighted histogram



1 bin corresponds to 1 orientation / angle

IV Keypoint Descriptor Extraction

Consider 16×16 window around keypoint
for each 16×16 " compute histogram.

Algorithm for Full descriptors.

1. divide 16×16 window into 4×4 grid

2. every block contains 4×4 pixels

3. compute orientation & mag.

4. Quantize orientation to 8 angles

5. weigh based on mag.

6. 4×4 cells $\times$ 8 orientation per cell = 128 dimensions for each keypoint/ features.

$\rightarrow$ 8 bin histogram

To normalise orientation to unit vector clip histogram. [remove values above 1]
 $\rightarrow$ normalisation is done to reduce effects of changes in illumination/ contrast.

Rotation invariant descriptor:

rotate each image patch according to dominant gradient.

then patches have canonical orientation.

$\downarrow$
refers to consistent orientation assigned to each keypoint to ensure descriptor is rotation invariant.

Rotation invariance: using canonical orientation ensures same feature in 2 diff images (one of which maybe rotated) will have the same/similar descriptors enabling reliable feature matching.

SIFT is robust to

- changes in view points / rotation
- changes in illumination

Advantage: fast & easy to implement.

Matching local features

- Can match keypoints of image to database/ keypoints of other image.
- look for similar patches in appearance.
- Measure similarity using Euclidean distance to measure diff. b/w feature vectors.
- take k -closest keypoints as match WRT query point.

Feature vector is a 128 dimensional descriptor that encodes local image gradients around a key point.

Which euclidean dist. indicates good match?

Ex: take top 5 matches & then have algo to choose one out of them

Use ~~RANSAC~~ + cost function for finding correct matching regions
RANSAC

↳ Random Sample Consensus

idea: iteratively select random subsets of data & fit a model to them.

then check how well this ^{model} fits to rest of data.

To add robustness to local matching

- consider ratio b/w 1st & 2nd best match in terms of euclidean dist.
- dist of query to 1st match / dist to 2nd match
- if ratio is low then best match is false.

Matching Against database.

Use visual words.

- similar descriptors correspond to objects with similar local context.
- quantise feature space by clustering features
- Visual word: group of image patches that lies on same cluster.
- Assign new visual word to a cluster by measuring distance to cluster centers.

Application: image/video retrieval.
can describe complete image by visual words ~~words~~

Chap 7 - Edges Lines & Blobs

Edge-detection transforms image from RGB pixels to set of curves at which image intensities change.

Edges occur at either object boundaries / at shadows.

Detect edges at places with strong discontinuities in the image intensities.

Discontinuities in image ^{brightness} occur at

- obj boundaries
- obj properties change
- discontinuities in depth / surface orientation
- illumination variations.

Requirements for good edge detector.

1. Good edge categorisation: edge v/s not edge

Find real edges & ignore noise/artifacts like shadows.

2. Good edge localization

- detect edge as close as possible to real image location.

Approaches for edge detection

Bottom-up: discontinuities in image intensities / obj boundaries

Top-down: continuity & closure, rely on existing knowledge.

CNN for edge detection

image $\rightarrow$ CNN $\rightarrow$ edges

Neural NW predicts the edges.

Bottom up & Top down:

i) detect edges at different scales with deep neural network. (bottom up)

ii) refine extracted edges with another deep NN-W. (top down)

Bottom-up: uses local pixel-info, detects edges based on image gradients and intensity changes.
no prior knowledge required [sobel/canny]

Top-down: starts with higher/previous knowledge of scene & uses that knowledge to guide detection of edges.

More computationally demanding. [template matching / Neural Net]

Gradient based edge detection: look for rapid change in intensities using gradients. Use gradient approximation based on 2D filters & convolute.

Challenge: noise could cause edges, strong noise leads to strong edge response.

Sol: smoothen the image [gaussian blur]

Standard edge detector

- 1) compute partial derivatives in x & y direction.
- 2) calc. gradient vector with discrete differentiation [Ex: sobel]
- 3) convolution operation: calc gradient based on neighbouring pixels.

Canny edge detector

- Works based on image gradients [strong changes in intensities]
- works well with step edges



step edge



ramp edge



roof edge

- robust compared to already seen approaches [1st & 2nd image derivatives / DoG / LoG / Hessian detector]
- multistage detection
- criteria
 1. low error rate
 2. Good localization
 3. single response: each edge is detected only once.

Steps:

1. convert image to grayscale
2. reduce noise [smoothing] $\rightarrow$ gaussian
3. intensity gradient calc.
 - sobel operator
 - mag & orientation per pixel
4. Non-maximum suppression $\rightarrow$ 2 additional steps
5. Hysteresis thresholding

Non Max suppression

- does edge thinning
- search for pixel with highest value in an edge in the gradient direction

hysteresis thresholding

- remove weak edges
- A high & low threshold
- strong & weak edges

what is it doing here?

Why is canny edge detector not used for SIFT?

Thresholding in canny removes many edges & info is lost if canny is used. Does it SIFT find stable keypoints [corners] for matching but canny is used for edges. Edges are not ideal for keypoint detection as they are less stable. Canny doesn't provide mechanism to assign orientation to corners.

Line Detection

By detecting lines we can recognise objects, scenes & etc. $\rightarrow$
Is edge detector enough for extracting lines?

Most objects defined by composition of lines.

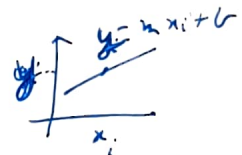
Edge detection does not work for line detection.
- cannot rely on edge detectors for lines.

Least square line fitting

Fits the best possible line to a given set of i/p points.

$D = \{(x_1, y_1) \dots (x_n, y_n)\} \rightarrow$ i/p data points [n in number]

goal minimize $C = \sum_{i=1}^n (m x_i + b - y_i)^2 \rightarrow$ error



$\Rightarrow$ we look for best m & b that explains i/p data D .

Matrix form

$$C = \sum_{i=1}^n ([x_i \ 1] \begin{bmatrix} m \\ b \end{bmatrix} - y_i)^2$$

$$= \sum_{i=1}^n \left(\begin{bmatrix} x_i & 1 \end{bmatrix} \begin{bmatrix} m \\ b \end{bmatrix} - y_i \right)^2 = \sum_{i=1}^n \|Xp - y\|^2$$

$$\Rightarrow p = X^{-1}y$$

$X^{-1} \rightarrow$ pseudo inverse of X .

$\rightarrow$ this least squares method is not robust to noise.

$\rightarrow$ inversion of X can be expensive.

How can we reach a robust sol. with outliers? $\rightarrow$ Can use RANSAC.

Voting for lines

voting is an approach to find best performing model out of a model set or best parameters for a specific family model.

- voting is based on particular features
- voting aims to collect more votes for the correct parameters of a model.
- Noise also casts votes but hopefully are not enough to dominate the process.

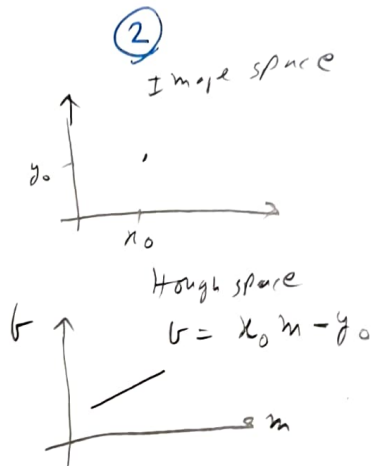
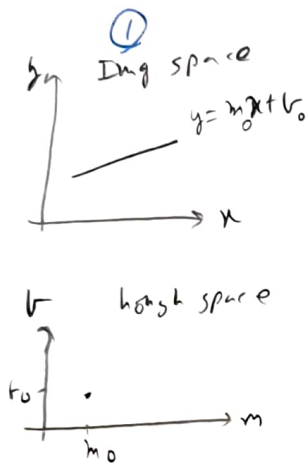
Methods: Hough transform & RANSAC

What is the sample space for voting?

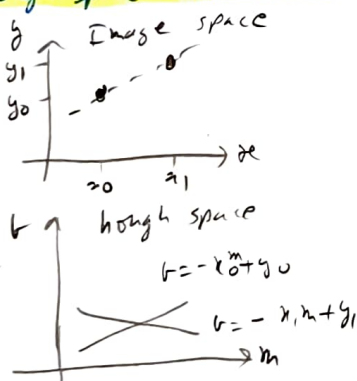
$\rightarrow$ Hough space

Hough space

- 1) Line in image space = point in Hough space
- 2) Line in Hough space = ~~line~~ point in image space.



Hough space line intersection



- 2 points in image space represented by 2 lines in Hough space
- if they intersect in Hough space the 2 points lie on same line. i.e. line formed by 2 points corresponds to intersection of 2 lines in Hough space
- By intersecting more lines in Hough space we can collect more votes for the dashed line in image space.

Challenges of Hough space: Vertical lines require infinite m but (m, b) space is bounded.
Sol: use polar representation for lines.

Polar representation



Point in image space is now a sinusoid in Hough space.

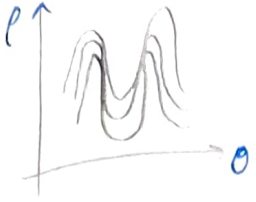
$\rho = x \cos \theta + y \sin \theta$ → if we know x & y what is left is integral of $d \theta$ i.e. a sinusoid.

if $d = \text{radius}$ then $\theta = [0, 360^\circ]$
if $d = \text{true/false}$ then $\theta = [0, 180^\circ]$

each point in (x, y) will now add a sinusoid in Hough space

$$\frac{x}{\rho \cos \theta} + \frac{y}{\rho \sin \theta} = 1 \rightarrow \text{eqn of line}$$

Voting is same with 6, m parameters i.e. (θ_i, ρ_i) with
max. votes form the line $\rho = x \cos(\theta_i) + y \sin(\theta_i)$



Algorithm

1. define values for θ & $\rho \rightarrow$ discretization comes into play here & affects the algorithm.
 2. " accumulator array H based on dimensions of θ & ρ
 3. detect edges on image I
 4. for each edge point (x, y) in image I
~~for $\theta = 0:180$~~ $\theta = \text{orientation at point}(x, y) \rightarrow$ red marking, here only if orientation is included in the Hough Transform.
 $\rho = x \cos \theta + y \sin \theta$
 $H(\theta, \rho) + 1$
 5. Look for combination (θ^*, ρ^*) that gives the max value in H
detected line: $\rho^* = x \cos(\theta^*) + y \sin(\theta^*)$
- $\rightarrow$ detecting more lines is possible by taking more maxima. Ex: top

3 maxima.

What is good discretisation for θ & ρ ?

coarse: many votes go to same bin / lesser lines detected

fine: ~~many~~ few votes go to same bin / more lines detected

$\rightarrow$ smoothing in accumulator array can improve detection.

Hough Transform for circles

Define circle with centre (a, b)

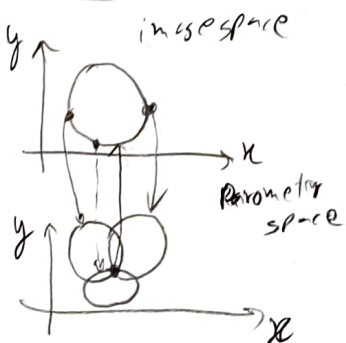
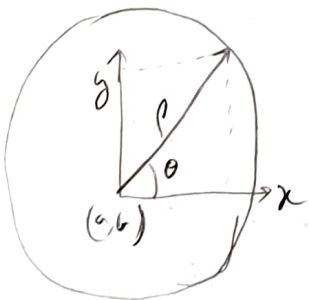
$$\Rightarrow x = a + \rho \cos(\theta)$$

$$y = b + \rho \sin(\theta)$$

$$\text{eqn of circle: } (x_i - a)^2 + (y_i - b)^2 = r^2$$

fix r & look for a, b

i.e. we fix the radius of a circle & look for circles of that radius in the image.



Algorithm:

1. Define accumulator array^H based on dimensions of a, b, P .
2. Detect edges in image I
3. For each edge point (x, y)

For each radius value P

For each value θ

$$a = x - P \cos(\theta)$$

$$b = y - P \sin(\theta)$$

$$H(a, b, P) \uparrow = 1$$

Notes on Hough space

- can cope with occlusion as all edge points vote independently.
- robust to noise due to voting scheme.
- multiple instance [line/circle] detection
- complex as parameter space increases. Ex: For 4 parameters & 1000 edges per param, we have $O(100^4)$
- Quantisation for binning creates errors. [discretisation]

Generalised Hough Transform [GHT]

- orig. hough tx used for analytical shapes like lines/circles
- GHT looks for the transformation parameters to map model into the image.
- GHT works for any shape.

Goal: match given template to edge data
Task: match template to edge data.

~~Algorithm~~ **Concept:** given a template

choose a center point inside shape.

$(x_c, y_c) \rightarrow$ reference point

$(x_{ij}, y_{ij}) \rightarrow$ set of landmark points
[edges, in a stable spatial arrangement]

Can ref point in GHT be outside the shape?

- Ans in 2 parts later

- Template consists of ref. point & set of landmark points.
- Define the **R-table** based on orientation of each landmark.
 L is a lookup table where orientation is the index.
Each cell of table stores gradient direction ϕ & the displacement vector $r = y - x$.



EX: R-table

i	ϕ_i	R_{ϕ_i}
1	0	$(r_{11}, a_{11}) (r_{12}, a_{12}) \dots (r_{1m}, a_{1m})$
2	$\Delta\phi$	$(r_{21}, a_{21}) (r_{22}, a_{22}) \dots (r_{2m}, a_{2m})$
...	$2\Delta\phi$	(r_{3k}, a_{3k})

How is disp vector r retrieved as 1 orientation corresponds to multiple disp vectors?

→ edge orientation ϕ_i indicates which row to pick up.

Now using ~~edge~~ ^{reference} image vote for the ref. point of R_e reference image (x_c, y_c) .

Intuition

- Learning [offline]

- Extract landmarks from an object to define its boundary points.
- for each boundary point calc. & store displacement vector & orientation in R table

- Test

- for each detected boundary point use orientation to access the R table & retrieve displacement vector r .
- Accumulate all disp. vectors & vote for max. location $[x_c, y_c]$ i.e. ref. point

$H[x, y, s, \theta]$

pos scale

orientation → vote for each of them & choose one with max. votes

Allows obj. detection by matching shape's boundary points to its ref. point using both orientation & spatial displacement info.

RANSAC - Random sample consensus

Finds subset that can lead to the correct solution.

- outliers can affect voting space & can cause wrong maxima votes.
- RANSAC computes model parameters considering the outliers.

Idea: Repeated random sub-sampling & assuming that there are sufficient inlier data-samples.

- RANSAC is outlier detection approach.

- " is **non-deterministic** i.e. gives better model parameters by running algo for more iterations.

Randomly subsample data set → estimate model parameters.

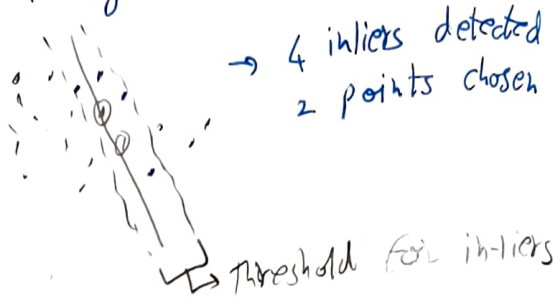
- if outlier is selected then line doesn't fit properly to the data

Algorithm:

1. Randomly select s points to estimate model parameters
2. Fit model to s points
3. Find inliers to fitted model
- thresholding t is necessary.
4. Repeat N times

Model with max inliers is finally selected.

Ex:



Parameters of RANSAC

- No. of samples
- No. of sampled points
- inlier threshold
- no. of iterations N

↳ can be set as function of desired success probability p .
 p → probability that RANSAC returns successful result after each run.

w → prob to select an inlier each time data is sampled

$$w = \frac{\text{N.O. of inliers}}{\text{No of samples}}$$

s → no. of sample points ~~required~~ for calc. model parameters

w^s → prob. of all points are inliers

$1 - w^s$ → " but atleast 1 point is outlier

Prob that RANSAC never selects inliers = $(1 - w^s)^N$

$$1 - p = (1 - w^s)^N \Rightarrow N = \frac{\log(1 - p)}{\log(1 - w^s)}$$

in practice calc. std. dev. of N & take multiple of it.

$$SD(N) = \frac{\sqrt{1 - w^s}}{w^s}$$

Adv

- works for any kind of param estimation

- empirically shown to work well most of the time.

Disadv

- requires parameter tuning & differs from prob to prob.
- assumes sufficient inliers, else can fail

Image segments

segments are better descriptors than edges.

Image edges are significant changes in intensity/color from adjacent pixels.
Image segments are contiguous regions of pixels sharing similar characteristics like color, texture or intensity. Techniques to segment: clustering, thresholding

Segmentation partitions the image into meaningful segments.

Types of segmentations

1. Binary seg.: background/foreground. separates obj of interest from rest of the image.
2. Semantic seg.: divides image into regions corresponding to specific obj.
3. Instance seg.: specific category (car1, car2...) assigns ID to diff obj. of same category.
4. Panoptic seg.: semantic & instance

Methods: Otsu-thresholding, clustering, region growing, deep neural networks
Energy-minimisation [Markov Random Fields], graph cut.

Edges

- low level features
- easy to obtain
- don't have to be closed
- partially describe objects

segments

- high level features
- complex to obtain
- they are closed
- completely describe objects.

Thresholding

I. Image binarization

Apply 1 threshold to the entire image.

$$I(x,y) = \begin{cases} 0 & \text{if } I(x,y) < T \\ 1 & \text{o.w} \end{cases}$$

Goal: select a threshold that creates large connected regions. Small connected regions should vanish. Can analyse the histogram of intensities to select.

II. Otsu thresholding

Uses gray-value histogram to find global threshold.

Goal: minimize the intra class variance.

Perform exhaustive search for threshold that is defined as weighted sum of 2 classes (A,B) variances.

$$\sigma_p^2(t) = P_A(t) \sigma_A^2 + P_B(t) \sigma_B^2$$

$P_A, P_B \rightarrow$ prob of 2 classes separated by threshold t .

$\sigma_A^2, \sigma_B^2 \rightarrow$ variance of 2 classes.

$$P_A(t) = \sum_{i=0}^{t-1} p(i)$$

$$P_B(t) = \sum_{i=t}^{L-1} p(i)$$

$L \rightarrow$ bins of histogram

for 2 class seg.

finding min intra-class var = max. interclass var.

$$\sigma_p^2(t) = P_A(t) P_B(t) [M_A(t) - M_B(t)]^2$$

$\downarrow$
class means

$$M_A(t) = \frac{\sum_{i=0}^{t-1} i p(i)}{P_A(t)}$$

$$M_B(t) = \frac{\sum_{i=t}^{L-1} i p(i)}{P_B(t)}$$

Algorithm:

1. compute histogram & probabilities for each intensity.
2. compute $M_A(t)$ & $M_B(t)$
3. $t=1 \dots$ max intensity
 update $M_A(t), M_B(t), P_A(t), P_B(t)$
 compute $\sigma_p^2(t)$ & pick min $\sigma_p^2(t)$.

How does n.o. of bins affect thresholding in otsu algorithm?

less bins

- means intensity range is divided into broader segments. simplifies hist. but smoothens out critical details.
- small variations get ignored
- less prone to noise.
- faster computations

More bins

- more detailed histogram allows precise thresholding.

- better seg. if image has finer details.

- more prone to noise
- more computationally complex.

Can ref. point in GHT be outside the boundary?

The ref. point can be outside the object boundary.

The algorithm uses the reference point as a pivot to accumulate votes in the Hough space, based on orientation & dist. of edge points.

Ref. point outside boundary may lead to

- i) increased computational complexity
- ii) reduced accuracy due to noise/clutter
- iii) potential false positives.

— x —

Chap 8 -

Geometry

Image alignment

- given 2 images what is the geometric relationship between them?
- As long as there are common objects on both the images we can learn the geometric relation b/w 2 images from the common objects/features. [Image mosaic]

Image filtering: $g(x) = h(f(x))$; $x \rightarrow$ position
 $g \rightarrow$ filtered o/p
 $h \rightarrow$ filters

- $h(\cdot)$ is implemented using convolution

- pixel values are affected from $h(\cdot)$

- Range of image is changed.

Image Warping: $g(x) = f(h(x))$

- $h(\cdot)$ corresponds to transformation Rat affects pixel coordinates not intensity/pixel values.

- Domain of image will be changed.

Parametric global warping:

Transformation T changes coordinate system

- it maps ~~$p(x,y)$~~ $p = (x,y)$ to $p' = (x',y')$

such that ~~$p = T(p')$~~ $p' = T(p)$

- T is global transformation because it applies the same for each pixel location. It is also described by fixed parameters.

Forward warping: each pixel goes to corresponding image location

for every (x,y) in $I(x,y)$

- compute destination location $p' = T(p)$ where $p' = (x',y')$

- copy $I(x,y)$ to new image/location $I'(x',y') = I(x,y)$

Inverse warping: each pixel at destination image is sampled from the source image

for over (x',y') in $I'(x',y')$

- compute source location $p = \hat{T}(p')$ where $p = (x,y)$

- resample $I(x,y)$ at location $p(x,y)$ A copy to

$I'(x',y') = I(x,y)$

What is the diff. b/w forward warping & inverse warping?

2D - linear transforms

We can define T to perform: scale, rotation, shear, mirror

We can define transformations as

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} a & b \\ c & d \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

2D Scaling

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} s_x & 0 \\ 0 & s_y \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

2D rotation around (0,0)

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

2D Shear

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} 1 & sh_x \\ sh_y & 1 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$



2D Mirror

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} -1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

2D Mirror over (0,0)

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} -1 & 0 \\ 0 & -1 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

Properties of linear transformations

1. Origin maps to origin
2. Lines map to lines
3. Parallel lines remain parallel
4. Ratios are preserved

5. closed under composition.

How do we treat translation?

Translation is also treated as a linear transform as it preserves points ($i \rightarrow j$).

Homogeneous Coordinates

~~With~~ with cartesian coordinates in euclidean space
- 11'e lines on same plane will not intersect at infinity.
- points at infinity are not meaningful in Euclidean space.

But in Projective geometry & projective space

- 2 11'e lines will meet at infinity ex: train rails meet at horizon
- Euclidean space is subset of projective geometry.

Method of transformation from 2D coordinates $\rightarrow$ homogeneous

- Approach to represent N-dimensional coordinates with N+1 elements

2D coordinates become homogeneous coordinates by adding w next to them.

$$\text{Ex: } (x_{\text{car}}, y_{\text{car}}) \rightarrow \underbrace{(x, y, w)}_{\text{homogeneous coordinates}}$$

$$x_{\text{car}} = \frac{x}{w}$$

$$y_{\text{car}} = \frac{y}{w}$$

w $\rightarrow$ is the scale factor

Examples

$$(4, 10, 2) \rightarrow \left(\frac{4}{2}, \frac{10}{2}\right) = (2, 5)$$

$$(8, 20, 4) \rightarrow \left(\frac{8}{4}, \frac{20}{4}\right) = (2, 5)$$

$(2, 5) \rightarrow (2, 5, 1)$
cartesian $\rightarrow$ homogeneous

Note: We scale homogeneous coordinates but in cartesian coordinates they remain same points.

$\rightarrow$ scalar product is same point in euclidean space

$\rightarrow$ homogeneous coordinates are scale invariant.

$$(x, y) \rightarrow (x, y, 1)$$

$$(x, y, w) \rightarrow \left(\frac{x}{w}, \frac{y}{w}\right)$$

$\rightarrow$ We need homogeneous coordinates to express non-linear transformations

2D-Affine transformations

They are a composition of linear and translation transformations.

- origin doesn't necessarily map to origin

- Lines map to lines

- Parallel lines remain parallel

- Point & planes are also preserved

- closed under composition.

Transformation T is defined as
$$\begin{bmatrix} x' \\ y' \\ w' \end{bmatrix} = \begin{bmatrix} a & b & c \\ d & e & f \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ w \end{bmatrix}$$

Translation

$$\begin{bmatrix} x' \\ y' \\ w' \end{bmatrix} = \begin{bmatrix} 1 & 0 & tx \\ 0 & 1 & ty \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ w \end{bmatrix}$$

Rotation

$$\begin{bmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Scale

$$\begin{bmatrix} sx & 0 & 0 \\ 0 & sy & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Shear

$$\begin{bmatrix} 1 & shx & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Affine transform: 1st translate & then rotate

$$\begin{bmatrix} x' \\ y' \\ w' \end{bmatrix} = \begin{bmatrix} \cos\theta & -\sin\theta & tx \cos\theta - ty \sin\theta \\ \sin\theta & \cos\theta & tx \sin\theta + ty \cos\theta \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ w \end{bmatrix}$$

Fitting an affine transform to keypoint matches

Given a set of keypoint correspondences over 2 image, we calculate/seek the affine transform.

- keypoints lie on the same plane

* i.e. $\begin{bmatrix} x' \\ y' \\ w' \end{bmatrix} \begin{bmatrix} x \\ y \\ w \end{bmatrix} \begin{bmatrix} tx \\ ty \end{bmatrix}$ are known, we search for $\begin{bmatrix} m_1 & m_2 \\ m_3 & m_4 \end{bmatrix}$

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} m_1 & m_2 \\ m_3 & m_4 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} + \begin{bmatrix} t_x \\ t_y \end{bmatrix}$$

How many matches do we need to compute transformation?

1 match is sufficient.

We need to solve 2 equations per feature match.

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} m_1 & m_2 \\ m_3 & m_4 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} + \begin{bmatrix} t_1 \\ t_2 \end{bmatrix}$$

eqns to solve look like

$$\begin{bmatrix} x_i & y_i & 0 & 0 & 1 & 0 \\ 0 & 0 & x_i & y_i & 0 & 1 \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \end{bmatrix} \begin{bmatrix} m_1 \\ m_2 \\ m_3 \\ m_4 \\ t_1 \\ t_2 \end{bmatrix} = \begin{bmatrix} x_i' \\ y_i' \\ \vdots \end{bmatrix}$$

2D Projective Transformations [Homography]

- Affine transformations with projective warps.

- l^le line do not necessarily remain l^le.

⊛ It is a mapping b/w 2 projective plane with same projection centre.

transformation is defined as

$$\begin{bmatrix} x' \\ y' \\ w' \end{bmatrix} = \begin{bmatrix} a & b & c \\ d & e & f \\ g & h & i \end{bmatrix} \begin{bmatrix} x \\ y \\ w \end{bmatrix}$$

Affine Transform

$$\begin{bmatrix} x' \\ y' \\ w' \end{bmatrix} = \begin{bmatrix} a & b & c \\ d & e & f \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ w \end{bmatrix}$$

- 6 degrees of freedom

- preserves parallelism

Projective Transform

$$\begin{bmatrix} x' \\ y' \\ w' \end{bmatrix} = \begin{bmatrix} a & b & c \\ d & e & f \\ g & h & i \end{bmatrix} \begin{bmatrix} x \\ y \\ w \end{bmatrix}$$

- 8 degrees of freedom

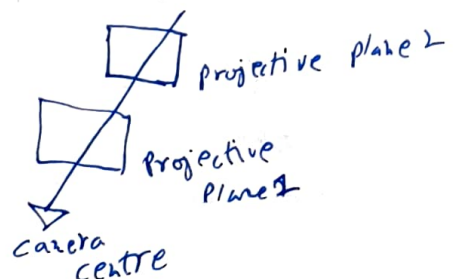
- preserves lines

- preserves lines

⊛ it is a mapping b/w 2 planes with same centre

$$\begin{bmatrix} wx' \\ wy' \\ w' \end{bmatrix} = \begin{bmatrix} h_1 & h_2 & h_3 \\ h_4 & h_5 & h_6 \\ h_7 & h_8 & h_9 \end{bmatrix} \begin{bmatrix} x \\ y \\ w \end{bmatrix}$$

$P' = H P$



Given a transform
how many
degrees of
freedom does
it have?

Image Mosaicing. [to create panoramic images]

- We have 2 images with camera rotation & no translation.
i.e. center of camera is fixed & only rotation is allowed.

Goal: to obtain a wider image by combining the image pair.

- can use multiple images
- Resulted mosaic is like a common plane for re-projecting all images.
- resulted mosaic can be seen as synthetic camera view.

stitching images together

I/p: sequence of ~~2~~ images from same position but different horizontal rotations.

step 1: Homography computation relying on 1st & 2nd image.

step 2: combine images to create mosaic

step 3: move to next image & repeat step 1 & 2 with last mosaic & the next image.

Note:

- Key points should be on same plane for homography.
- Use RANSAC to filter bad points.

Homography computation for image mosaicing.

Pair of corresponding image points required for computing homography H .

$$p' = Hp$$
$$\begin{bmatrix} wx' \\ wy' \\ w' \end{bmatrix} = \begin{bmatrix} h_1 & h_2 & h_3 \\ h_4 & h_5 & h_6 \\ h_7 & h_8 & h_9 \end{bmatrix} \begin{bmatrix} x \\ y \\ w \end{bmatrix}$$

Assuming $h_9 = 1$ for up to scale transformations. We have 8 unknowns $[h_1, \dots, h_8]$ to find.

applying homography.

1. compute $p' = Hp$ & convert from p' to euclidean coordinates

2. Sample pixel value by

i) Forward warping: send pixel value from left image to corresponding location of right image.

ii) Inverse warping: get location as $p = H^{-1}p'$ & sample pixel value.

Describe pseudo algo for feature matching?

Describe pipeline for forming mosaic/panorama?

Pin-hole camera model

single aperture camera without lens.

↳ an opening/hole through which light passes.

Light rays travel through pinhole & project an inverted image on the image plane.

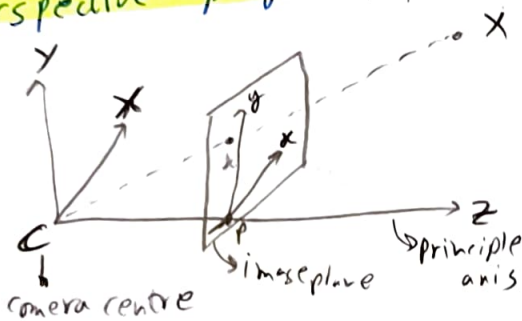
Camera matrix $[3 \times 4]$ ~~map~~ matrix maps the 3D world to coordinates of image plane. It is made of

- i) intrinsic parameters
- ii) extrinsic parameters

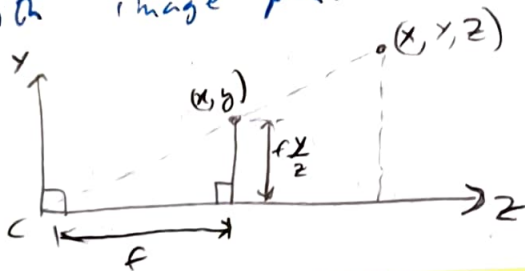
Camera calibration is the process of obtaining these parameters.

Perspective projection model.

Normalized camera coordinate system



- Principal axis, as well as X, Y axis of camera are parallel with image plane axis u, v .



- Principle/optical axis: axis pointing to camera viewing direction.

- Image plane is the projection of the 3D world.

- camera centre (C) is the origin of the coordinate system.

$f \rightarrow$ focal length
the distance at which the image plane is located at from the origin C.

$$u = f \frac{x}{z} ; v = f \frac{y}{z}$$

homogeneous coordinates for perspective projection model:

$$\lambda \begin{pmatrix} x \\ y \\ 1 \end{pmatrix} = \begin{pmatrix} f x \\ f y \\ z \end{pmatrix} = \begin{bmatrix} f & 0 & 0 & 0 \\ 0 & f & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix}$$

homogeneous coordinates of 2D image point x

camera projection matrix

homogeneous coordinates of 3D image point X .

Camera calibration

COS $\rightarrow$ coordinate system

refers to determining transformation from world COS to camera COS.

$$\begin{bmatrix} 2D \text{ point} \\ 3 \times 1 \text{ vector} \end{bmatrix} = \begin{bmatrix} \text{camera 2 pixel coord. transfr.} \\ 3 \times 3 \text{ matrix} \end{bmatrix} \begin{bmatrix} \text{canonical projection matrix} \\ 3 \times 4 \end{bmatrix} \begin{bmatrix} \text{world 2 camera coord system transfr.} \\ 4 \times 4 \text{ matrix} \end{bmatrix} \begin{bmatrix} 3D \text{ point} \\ 4 \times 1 \text{ vector} \end{bmatrix}$$

intrinsic camera parameters extrinsic camera parameters

Intrinsic Parameters

- focal length
- principal point coordinates
- pixel scaling factors
- skew
- radial distortion

Extrinsic Parameters

- Rotation A
- translation b/w world COS to camera COS.

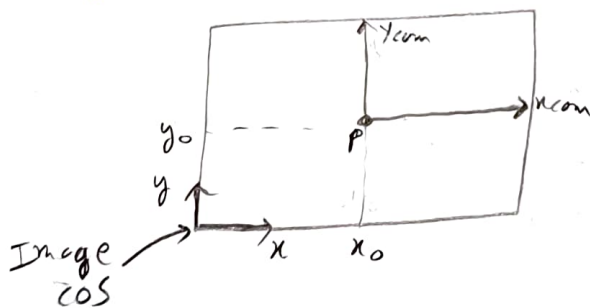
$$x = K \cdot [I | 0] \cdot \begin{bmatrix} R & t \\ 0^T & 1 \end{bmatrix} \cdot X$$

intrinsic calib. matrix canonical projection matrix extrinsic param.

rotate & translate

$$P = K [R | t] \rightarrow \text{general camera projection matrix}$$

- Extrinsic param: refer to transformation from world COS to camera COS.
- Principal point is the point of intersection of principal axis and the image plane
- Image origin is at same principal point for normalised camera.



- in image COS normally image origin is in the top left corner

Principal point mapping

- Map principal point to (p_x, p_y) instead of $(0, 0)$.

$$x = f \frac{X}{Z} + p_x$$

$$y = f \frac{Y}{Z} + p_y$$

$$\lambda \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \begin{pmatrix} fX + ZP_x \\ fY + ZP_y \\ Z \end{pmatrix} = \begin{bmatrix} f & 0 & P_x & 0 \\ 0 & f & P_y & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \begin{pmatrix} X \\ Y \\ Z \\ 1 \end{pmatrix}$$

$$\downarrow$$

$$P = K [I | 0]$$

$$\begin{bmatrix} f & 0 & P_x & 0 \\ 0 & f & P_y & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} f & 0 & P_x & 0 \\ 0 & f & P_y & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \rightarrow \text{intrinsic matrix}$$

$$M_{int} = [K | 0]$$

$\downarrow$ $K \rightarrow$ intrinsic calib. matrix

principal point & focal length: world units, e.g. millimeters.

What unit do we use for image coordinate?

- scaling factor for converting world unit to pixels.

m_x pixels/mm in horizontal direction
 m_y pixels/mm in vertical direction

plotted here
to scale for λ

$$\begin{bmatrix} m_x & 0 & 0 \\ 0 & m_y & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Scaling factors

$$\text{pixel size} = \frac{1}{m_x} \times \frac{1}{m_y} \text{ (mm)}$$

$$\begin{bmatrix} f & 0 & P_x \\ 0 & f & P_y \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} a_x & 0 & b_x \\ 0 & a_y & b_y \\ 0 & 0 & 1 \end{bmatrix}$$

Calib. matrix
K in metric units

Calib. matrix
K in pixel units.

Image coordinates $u = \begin{bmatrix} u \\ v \end{bmatrix}$ $\xleftarrow{\text{perspective projection}}$ camera coordinates $x_c = \begin{bmatrix} x_c \\ y_c \\ z_c \end{bmatrix}$

M_{int} $\xleftarrow{\text{perspective projection}}$ x_c

extrinsic $\uparrow$ coordinate transformation

$x_w = \begin{bmatrix} x_w \\ y_w \\ z_w \end{bmatrix}$

Intrinsic Parameters

- Rotation & translation to relate world cos with camera cos.

Non-homogeneous coordinates

$\rightarrow$ rotation matrix

$$\tilde{x}_{cam} = R(\tilde{x} - \tilde{c}) = R\tilde{x} + t$$

Normalised camera coordinates

$\rightarrow$ world cos to camera cos.
 $\downarrow$ camera center coordinates in world cos.
 Point coordinates in world cos.

homogeneous coordinates

$$\begin{bmatrix} \tilde{x}_{cam} \\ 1 \end{bmatrix} = \begin{bmatrix} R & t \\ 0^T & 1 \end{bmatrix} \begin{pmatrix} \tilde{x} \\ 1 \end{pmatrix}$$

4 x 4 transformation matrix

$$\rightarrow x_{cam} = \begin{bmatrix} R & t \\ 0^T & 1 \end{bmatrix} x$$

$$M_{ext} = \begin{bmatrix} R & t \\ 0^T & 1 \end{bmatrix} = \begin{bmatrix} r_{11} & r_{12} & r_{13} & t_x \\ r_{21} & r_{22} & r_{23} & t_y \\ r_{31} & r_{32} & r_{33} & t_z \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Calibration Process

Is process of finding the projection matrix.

$$P = \begin{bmatrix} p_{11} & p_{12} & p_{13} & p_{14} \\ p_{21} & p_{22} & p_{23} & p_{24} \\ p_{31} & p_{32} & p_{33} & p_{34} \end{bmatrix} \rightarrow \text{find these 12 numbers.}$$

Goal of calibration is to estimate intrinsic & extrinsic camera parameters.

- Use calibration pattern like checkerboard where we can extract 2D & 3D coordinates.
- Linear approach estimates projection matrix P but not M int & M ext.
- Use QR factorisation for rough estimate of K, R, t .

Pattern-based calibration

Assumptions

1. We assume planar calibration pattern.
2. Correspondences b/w 3D points & their 2D image plane projections.

This relation is expressed as

$$X_i = \begin{pmatrix} x_i \\ y_i \\ z_i \\ 1 \end{pmatrix} \rightarrow \text{3D point}$$

$$x_i = \begin{pmatrix} x_i \\ y_i \\ z_i \end{pmatrix} \rightarrow \text{Pixel coordinates}$$

$$(u_i, v_i) = \left(\frac{x_i}{z_i}, \frac{y_i}{z_i} \right)$$

$\begin{matrix} p_1 \\ p_2 \\ p_3 \end{matrix} \rightarrow 3 \text{ rows of projection matrix}$

$$u_i = \frac{p_1 x_i}{p_3 x_i}$$

$$v_i = \frac{p_2 x_i}{p_3 x_i}$$

$$x_i = p_1 x_i$$

$$y_i = p_2 x_i$$

$$z_i = p_3 x_i$$

$$\Rightarrow \begin{cases} u_i(p_3 x_i) - (p_1 x_i) = 0 \\ v_i(p_3 x_i) - (p_2 x_i) = 0 \end{cases} \rightarrow \begin{matrix} 2 \text{ lin eqns} \\ \text{for each} \\ \text{correspondence} \end{matrix}$$

- Projection matrix has 11 unknowns, $[p_{34}=1]$
so we need 6 pairs of correspondence points b/w 3D-2D world.

Planar Homography Solution

done in (every) day life

$$\begin{bmatrix} x_i' \\ y_i' \\ 1 \end{bmatrix} = \begin{bmatrix} f & s & p_x \\ 0 & f & p_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} R & t \\ 0^T & 1 \end{bmatrix} \begin{bmatrix} x_i \\ y_i \\ z_i \\ 1 \end{bmatrix}$$

uses 2D pattern

$$= \begin{bmatrix} f & s & p_x \\ 0 & f & p_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} r_{11} & r_{12} & r_{13} & t_1 \\ r_{21} & r_{22} & r_{23} & t_2 \\ r_{31} & r_{32} & r_{33} & t_3 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_i' \\ y_i' \\ z_i' \\ 1 \end{bmatrix}$$

general calculation

$$= \begin{bmatrix} f & s & p_x \\ 0 & f & p_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} r_{11} & r_{12} & r_{13} & t_1 \\ r_{21} & r_{22} & r_{23} & t_2 \\ r_{31} & r_{32} & r_{33} & t_3 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_i' \\ y_i' \\ z_i' \\ 1 \end{bmatrix}$$

Why do we remove a few elements?
→ simplification because of a 2D pattern used.

$$= \begin{bmatrix} f & s & p_x \\ 0 & f & p_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} r_{11} & r_{12} & t_1 \\ r_{21} & r_{22} & t_2 \\ r_{31} & r_{32} & t_3 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_i' \\ y_i' \\ z_i' \\ 1 \end{bmatrix} = \begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & h_{33} \end{bmatrix} \begin{bmatrix} x_i' \\ y_i' \\ z_i' \\ 1 \end{bmatrix}$$

homography matrix

Non-linear approach

1. define objective func - intrinsic or extrinsic parameters
re-projection error: $\sum_i \| \text{proj}(K[R|E]X_i) - x_i \|_2^2$
→ projecting x_i to image plane using K, R & t parameters.

2. Additional constraints such as focal length, visibility points, or homography
A radial distortion can be included as parameters.

3. Minimise the obj. func. with non-linear optimiser ex: Levenberg-Marquardt

Triangulation: given projections of 3D point in 2 or more images reconstruct 3D point coordinates.

Geometric

- each view generates a visual ray based on corresponding projection.
- goal: intersect rays for reconstruction of 3D points.
- due to noise rays don't meet exactly
- sol for 2 views: find shortest segment connecting 2 rays & set mid point as X_i .
- for more views apply same on all possible pairs.

Optimisation approach

- define obj func & solve using non-linear optimiser

obj for 2 views:

$$\| \text{proj}(P_1 X_i) - x_i \|_2^2 + \| \text{proj}(P_2 X_i) - x_i \|_2^2$$

triangulation for structure from motion

- structure: given camera parameters, calc. 3D coordinates
- Motion: given 3D point coordinates, compute camera parameters & do triangulation.

→ do both simultaneously to improve performance

Chap 10 — Detection

1) **Object detection**: Aims to detect objects in an RGB image. The number of objects is not known in advance.

2) **Semantic segmentation**: Detects for each pixel the object category it belongs to. All object categories must be known in advance.

- For a new object it can be erroneously classified / can be categorised as background. ^{used for} ~~detected~~: classification + localisation

3) **Instance segmentation**: Identifies for each pixel the object instance it belongs to, as well as each instance for every object category. All object categories must be known in advance. ^{used for}: object detection

OpenSet learning can add new objects for detection. ^{→ single shot detector}

Example of few object detector: YOLO, SSD
↳ you only look once

object detection Problem

i/p: RGB Image

o/p: 2D bounding boxes with category labels & confidence score.

challenge: No. of objects in image is not known in advance.

Classification & regression are 2 ML problems to solve for object detection.

Object detection aims to detect objects in visual modalities such as images or point clouds.

Framework:

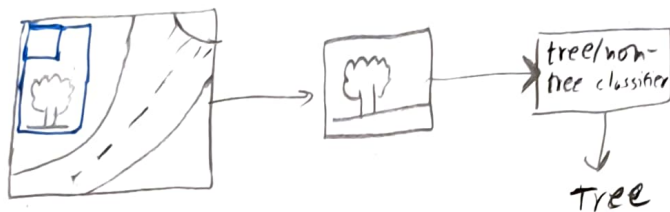
- 1) Build/train object model
- 2) generate candidate regions for given image
- 3) Score each region.

How is model defined?

How is region defined?

Window-template-based models

Given the image representation train a binary classifier to classify the region.



Use sliding window for obtaining candidate scores.

optimisation: Use Non-max. suppression to throw away few boxes regions.

disadvantage

- not scale invariant
- not scalable / expensive as we have to run at different scales.

What do we modify to obtain different scales?

- scale the image.

Pipeline:

- Pipeline not used anymore: why?

- features have to be handcrafted
- design of classifier itself is complex

Neural nets are better as classification & feature detection is done in same step.

Training:

1. gather & annotate training image data
2. Extract features (ex: hand crafted features like histogram of gradients)
3. Define the classifier (ex: Support Vector Machine, Random Forest)

Test:

1. Run sliding window of fixed size over the RGB image.
2. Extract features from each window (image crop)
3. Classify features from each crop with trained classifier.
4. Perform Non-Max. suppression.

challenges:

- 1) Image should be captured from diff. view points.
- 2) Many objects are articulated hence not well captured from single image crop.
- 3) Occluded objects will not return a high response score. They need special treatment.

sol: part based models

Evaluation Metrics for obj detection.

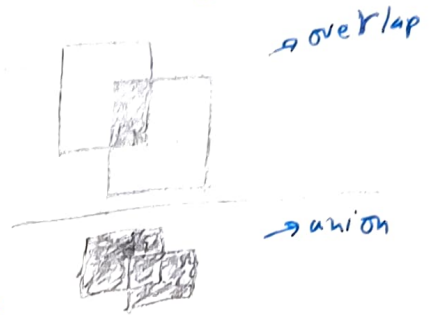
- Intersection over Union (IoU)

$$IoU = \frac{\text{Area of overlap}}{\text{Area of union}}$$

overlap b/w ground truth bounding box & predicted bounding box.

- successful detection when $IoU > 0.5$

Problem: object might not fit into bounding box very well.



- True Positive: No. of objects correctly detected ($IoU > 0.5$)

i.e. overlap > 50%

- False Positive: wrong detection

- False -ve: No. of objects not detected ($IoU < 0.5$)

$$mAP = \frac{1}{\text{classes}} \sum_{c \in \text{classes}} \frac{TP(c)}{TP(c) + FP(c)}$$

Feature extraction

Histogram of Oriented Gradients (HOG)

Approach: Represent image patches with histograms.

Features to account for viewpoint variance & illumination changes.

orientation: 9 bins

Histogram in 8×8 pixel cells.

output HOG encodes contours / shape [refer image of face in pg 18, chap 10]

- HOG features are invariant to image deformations such as rotation, translation & scale.

- HOG is a feature descriptor that replaces the pixel representations.

- Classifier is trained on HOG features to give object categories.

What are 2 alternative feature representations to replace HOG?

- USE SIFT features.

Feature rep we learnt

HOG + SVM = classical method of object detection

- SIFT for edge features

- SURF - speed up Robust Features

- Canny edge detector

- HOG

- LBP [Local Binary Pattern]

- Gabor filters for texture features

Part-based Models

- Idea: define an object as a collection of parts.
1. Model each part based on appearance & spatial configuration.
 2. learn individual appearance model of each body part.
 3. learn spatial relation b/w diff. parts.

- Learning spatial config. refers to extracting part-based templates.

- During testing part based template is used to obtain a score
- A threshold on score is necessary to decide for obj vs non-obj.

Part-based model: model articulated objects with appearance & spatial/viewpoint changes.

↳ defines object as constellation of parts with learnable configuration & appearance.

Filter/feature map

Root filter: roughly represents the appearance of the object.

Part filter: local object appearance. Covers parts of the object.

Spring connection (b/w parts): spatial relation for pairs of parts.

part filters are twice the resolution of a root filter.

↳ scores the part filter location relative to root.

Human body Pose estimation

Human body: Defined as a set of keypoints that forms a skeleton when connected. Also possible to define body parts with position width & height.

i/p: image, depth/point cloud.

Pose estimation

- body pose can be represented by a graph where nodes indicate body parts/keypoints & edges connectivity.

goal: model the body kinematics.

Benefits:

- occluded object can be detected
- generalised for object type

2D-pose estimation

single view single human: 1 individual in the image. Also goal is to estimate 2D pose from 1 image.

single view multiple human: Multiple individuals in image, where number is unknown in advance. goal: estimate 2D pose.

3D pose estimation

Multiview single human: Single individual in ^{multiple} images/camera views. system calibrated & synchronised in time. goal: estimate 3D body pose from set of images

Multiview Multi human: multiple individuals in multiple views. Number is unknown. goal: estimate 3D body pose of each individual.

Methods for human pose estimation

holistic approach

- pose inferred at one step
- body parts then available
- **Top down** approach
- more complex

Part based approach

- body parts 1st detected, followed by estimation of body pose.
- **bottom up** approach.
- better approach.

What is limitation of holistic/part based approach?
Which method is more complex?

state space generation

hypothesis generation:

- 2D part detection i/p
- Triangulation
- combination of all view pairs

- object detection in CV significantly influenced by CNN.

Before CNN

- sliding window & part based object detection

After CNN

- detection with region proposal approaches: RCNN, Fast RCNN, Faster RCNN, ComplexNet or SSD.

current state-of-art obj. detection based on CNNs.

Obj. detection with CNN

Recognition: refers to classification of an image/image crop.

- classify image category given the feature representation of the image
- classification of image crop can be done using bounding boxes & sliding window technique.
- bounding boxes classified into object category or to background.

Challenges:

1) Too many bounding boxes can occur during detection

sol 1: **Quantise location of bounding boxes.** can be done using a grid where each cell includes bounding boxes of different scale.

sol 2: learn to correct quantisation error.

Ground truth bounding box might not always match quantized bounding box $\therefore$ we **redefine ground-truth.**

Why not use any other shape for bounding boxes?

- annotation is more difficult.

2) Too many bounding box predictions during testing

sol: use non-maximum suppression to reduce number of boxes

3) Too many background boxes: Highly imbalanced foreground & background relation.

sol 1: design objective function during learning that balances foreground & background boxes.

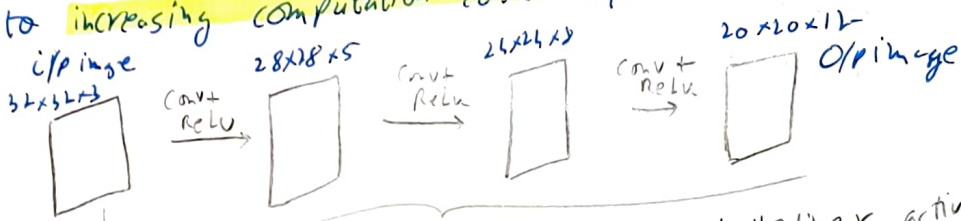
Alternative: sol 1: cascaded model i.e. concatenation of diff. models. (multi-stage model)

CNNs Chap-11

- CNN is a composition of convolution operations of multiple filters combined with activation functions & other differentiable operations.
- CNN approximates a function to map i/p image to some o/p.
Ex: obj category.
- CNN has several layers of convolution & each layer consists of several filters.

Note:

- every kernel generates a feature map.
- CNN is natural way of processing images.
- for 1 image we can have many kernels & hence CNN works best as it has several layers.
- Convolution used to learn image features.
- Always combined with non-linear activation functions.
- Due to increasing computation cost i/p is gradually downsampled.



CNN: composition of linear & Nonlinear activations
We use non-linear activations because problem is non-linear.

every kernel approximates a function/
" " " " detects diff. features.

R-CNNs [Region-based CNNs]

- step 1: collect Region of Interests. i.e. identify number of bounding box object region candidates, using selective search.
- step 2: extract features for each ROI
- step 3: classify the extracted features.

Pipeline

1. Pre-train CNN on image classification
2. Extract ROI with selective search
3. Warp ROI to fixed size
4. Fine tune pre-trained CNN on warped ROIs for object classification.
5. Given an image, extract feature vector & train SVM for each class independently.
6. Train regression model to improve predicted region window.

Limitations:

- 1) NMS [non-max-suppression] reduces number of bounding boxes, but still remains large.
- 2) Approach is complex as it involves 3 different models.
What are the 3 different models?

Fast R-CNN

- Unify all models of R-CNN to single model. [shift all models to Neural nets & remove SVM & regression]
- Single CNN used to perform all tasks.
- RoI pooling to standardize the proposal windows.

Pipeline

1. Pre train CNN on image classification
2. Extract RoIs with selective search. [proposal method]
3. Add RoI pooling layer for region standardization.
4. Branch the model to object classification & bounding box regressor.
use soft max classifier for classification.

What is RoI pooling?

RoI pooling is used to extract fixed size feature maps from varying sized RoIs. making it possible to feed them into fully connected layers for classification & Box regression.

Limitations

- 1) Faster than R-CNN but is still slow
- 2) Region proposals generated with another approach. [Major limitation]

Faster R-CNN

- integrates region proposal algo. into CNN model.
uses a single model that generates region proposals with RPN (region Proposal Network)

Pipeline:

1. Pretrain CNN on image classification
2. Fine-tune RPN
3. Train fast RCNN with proposals of RPN
4. Train RPN based on fast RCNN. Fine-tune only RPN layers

5. Fine-tune fast RCNN layers

6. Repeat 4 & 5

standard Datasets

MNIST database: Image recognition dataset with handwritten digits.

CIFAR 10 & 100 dataset: closer to real world images

Canadian Institute For advanced Research.

PASCAL VOC: for several CV tasks. [classification & detection]
Action classification
semantic segmentation
categorical seg.

Visual Object Classes

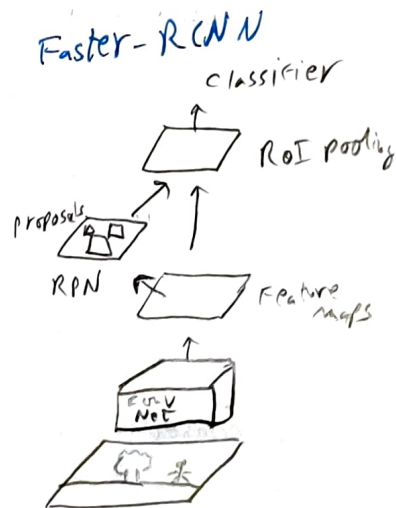
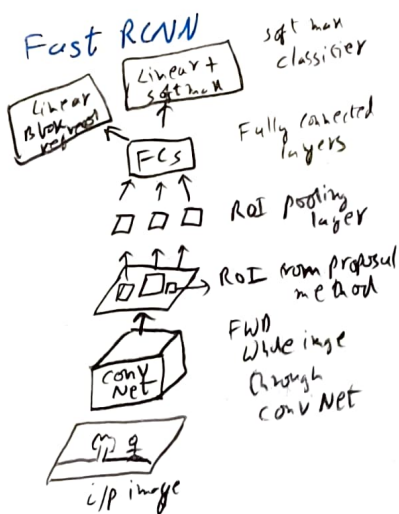
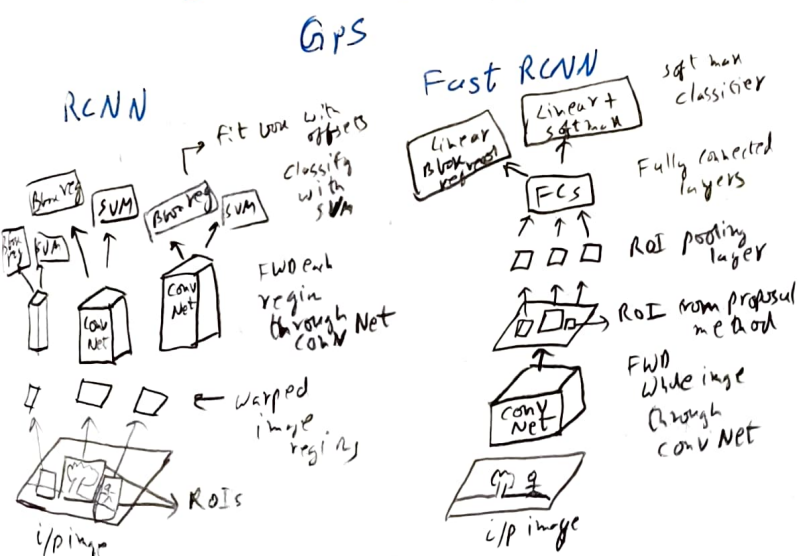
Image Net: closer to real world images.
- dataset based on WordNet hierarchy. [object localisation]
categorical seg.

COCO dataset: recognition in context, object detection, segmentation
A human pose estimation, super-pixel stuff segmentation.
categorical seg.

Youtube bounding boxes: Large scale video dataset with single-object
Bbox annotations.

KITTI: Real world computer vision benchmarks.
- motivated for automated driving
2 video cameras, Velodyne laser sensors
GPS

[stereo & flow estimation,
obj tracking
semantic seg.
instance seg.]



Chap 12 — Image segmentation

Applications: scene understanding, medical image analysis & robotic perception

classical methods: thresholding, histogram based binning or K-means clustering.

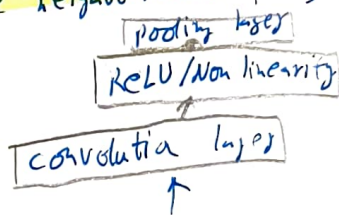
Now: end-to-end optimised deep learning models which learn features from data.

CNN: weights & filter kernels learned by minimising loss function on set of labeled training data.

Typical CNN layer

consists of

- several convolutions in ll'e to produce linear activations
- Non linear activation function [ex: rectified linear unit (ReLU)]
- Pooling summarises response over whole neighbourhood. → next layer
ex: max pooling makes features approximately invariant to small translations.



semantic seg. performs pixel-level labeling with a set of object categories.

Categoric/panoptic → semantic instance seg.

→ umbrella term & semantic seg. is one of the methods.

→ aims to divide image into distinct categories region or objects each belonging to specific category/class.

U-Net

- Trade off: hierarchical features from different scales v/s accurate localization
- Idea: capture context by reducing spatial dimensions & subsequently restore spatial dimensions. [contraction followed by expansion]
- skips connections: direct connections from contracting to expanding path.
- Preserve high-resolution layers of contracting path
- 2 3×3 convolutions [unpadded]
- ReLU
- 2×2 max pooling operation with stride 2 for down sampling.
- each down sampling step doubles n.o. of feature channels.

Layers of expanding path

- upsampling
- 2×2 convolutions, doubling of feature channels.
- concatenation with feature map from contracting path
- 2 3×3 convolutions each followed by ReLU.

Final layer: 1×1 convolution to map 64-dim. feature vector to desired n.o. of classes.

DeepLab V3

Atrous or dilated convolution: upsample the filter kernels, equivalent to inserting zeros or holes between filter weights.

Atrous rate r : insert $r-1$ zeros b/w filter weights

Output stride: ratio of i/p spatial resolution to output resolution.

Atrous convolution introduces holes b/w weights of the filter allowing the network to expand its respective field without increasing number of parameters or losing spatial resolution.

Ex: for a 3×3 kernel
normal convolution

a	b	c
d	e	f
g	h	i

uses 3×3 area around center pixel

Atrous conv. with rate 2

a	o	b	o	c
d	o	e	o	f
g	o	h	o	i

uses same 3×3 filter but looks at 5×5 region.

→ enlarge field of view of filters to incorporate multi-scale context.

→ control resolution of features & increase content without additional parameters. [→ Atrous conv. allows DeepLab V3 to maintain spatial resolution of feature maps. Usually downsampling operations like pooling/strided conv. reduce resolution of feature maps, leading to loss of fine details in tasks like seg.]

→ atrous conv. increases receptive field without adding extra weights or parameters (i.e. capture more content from image)

By adjusting the dilation rate, atrous conv. increases size of receptive field, i.e. the filter sees larger area of i/p, without needing larger filter/more computations.

→ objects in images can appear at various scales. Ex: car which is close to camera appears large but small if away from camera. DeepLab V3 captures ~~more~~ info at diff. scales by applying atrous conv. with different dilation rates. This multi-scale processing allows model to understand both small & large-scale structures.

Pooling in DeepLab V3

Atrous spatial pyramid pooling [ASPP]

- 4 11×11 conv. with different atrous rates are applied on feature map.

- This helps in capturing multiscale information
- Also improves precision & robustness of seg. models.

1. global average on last feature map of the feature extractor

GAP [Global average pooling] computes average of all values in each feature map, reducing feature map of spatial dim. into a single value per feature channel.

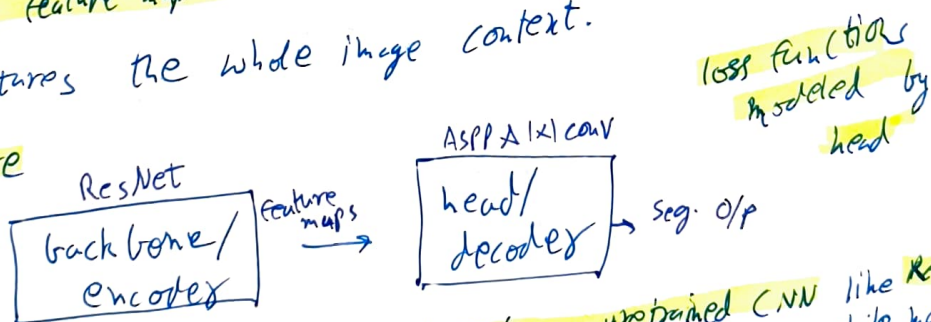
2. 1×1 conv.

after GAP resulting vector is processed by 1×1 conv. which is seen as linear transformation that adjusts feature representations (i.e. it modifies feature map channels w/o changing spatial dim.) in this case 1×1 .

3. Bilinearly upsample feature map to desired spatial dim.
After 1×1 conv. the resulting 1×1 feature map is bilinearly upsampled to match spatial dim. of orig. feature map.
Bilinear upsampling is an interpolation technique to increase size of feature map while maintaining transitions b/w pixel values.

→ This ^{ASPP} captures the whole image context.

Architecture



Backbone: feature extraction part of network often a pretrained CNN like ResNet, which is modified with atrous conv. to extract multi-scale info while maintaining resolution.

Encoder: part of model that downscales & encodes input image into set of feature maps, capturing high level abstract representation.

head: module responsible for processing extracted features (via ASPP & 1×1 conv.) & producing final seg. o/p.

Decoder: bilinear upsampling used to restore resolution after ASPP module processes the features.

Drawback: precise spatial info related to obj boundaries missing due to pooling & strided conv. in feature extractor.

DeepLab V3+

→ extend DeepLab V3 by a decoder module that refines seg. results along obj boundaries.

→ combine advantages of ASPP & UNet.

Depthwise separable conv.

Depthwise separable conv. = Depthwise conv. 1st + followed by pointwise conv.

Depthwise conv: spatial conv. individually for each i/p channel.

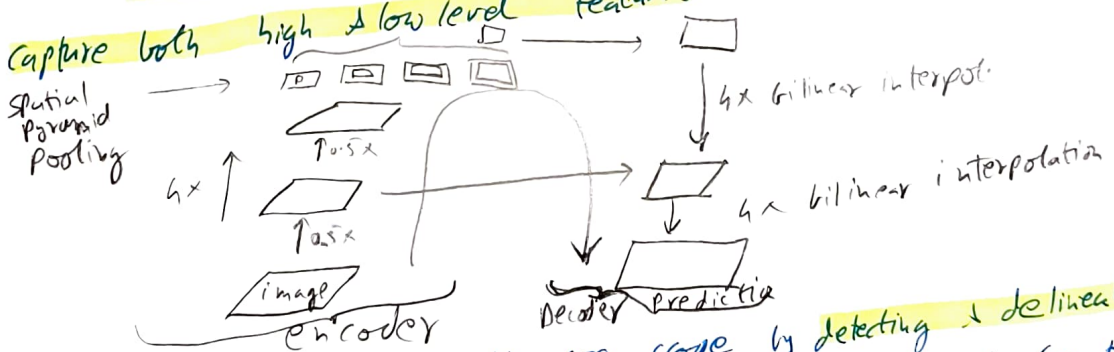
Pointwise conv: combine o/p of depthwise conv.

→ reduces computational complexity.

Encoder: extract high-level semantically rich features at low spatial resolution.

decoder: combine encoder o/p with low level, but higher resolution intermediate features from feature extractor.

→ Capture both high & low level features for improved seg. boundaries.



Instance seg: extends semantic seg. scope by detecting & delineating each object of interest in image. COCO dataset can be used for this task.

Mask RCNN for Instance seg.

Faster RCNN with 3 output branches

1. compute bounding box coordinates

2. classify object in bounding box

3. generate binary mask to segment object

→ loss function combines losses of Bbox coordinates, predicted class & segmentation mask.